GenAI Course Notes – July 2026

Sessions 9–15 · in date order · each session starts with its date page

DATE	SESSION & TOPIC		PAGE
4 July 2026	**Session 9**	Deep Learning Fundamentals	2
5 July 2026	**Session 10**	CNNs, Transfer Learning & RNNs	14
11 July 2026	**Session 11**	Transformers & BERT	26
12 July 2026	**Session 12**	Fine-Tuning: PEFT, LoRA & QLoRA	37
19 July 2026	**Session 13**	RAG Evaluation Metrics	47
25 July 2026	**Session 14**	Low-Code RAG on Azure	58
26 July 2026	**Session 15**	Graph RAG & Corrective RAG	67

SESSION 9

Saturday, 4 July 2026

Deep Learning Fundamentals

Deep Learning Notes – Session 9 (4 July 2026)

Oct 14, 2026 · @VIjay

1. Why deep learning now

This session gives the deep-learning background needed to understand transformers and fine-tuning. GenAI interviews rarely ask DL coding, but you should know the vocabulary (MLP, CNN, LSTM, backprop, vanishing gradients) and where each model fits.

Course roadmap from here: deep learning basics → Transformer ("Attention Is All You Need") → fine-tuning (PEFT, LoRA, QLoRA; Hugging Face, OpenAI) → tool/function calling, MCP, A2A, AI gateway → observability, guardrails, responsible AI → Graph RAG and CRAG.

Learning advice from class: practise each use case 3–4 times at intervals (after 15 days, a month) on new data. Don't try to read all of the LangChain docs; focus on what's taught. Python basics (NumPy, Pandas, loops, classes) are enough for most GenAI interviews.

2. Machine learning vs. deep learning

AI ⊃ ML ⊃ deep learning. ML learns patterns from data instead of fixed rules (e.g. house price from size, location, age, bedrooms: y = wx + b). Deep learning uses neural networks with many **hidden layers** ("deep").

#	Aspect	Machine learning	Deep learning
1	Approach	More manual: you design the features	More automatic: the network learns features
2	How it learns	Patterns from data via algorithms (trees, regression…)	Patterns via many layers of neurons
3	Data needed	Small to medium	Large
4	Feature engineering	By hand (e.g. combine Titanic siblings + parents into "family size")	Learned in hidden layers
5	Software	scikit-learn, NumPy, Pandas, Matplotlib, Seaborn	TensorFlow, Keras, PyTorch (plus NLTK/spaCy for text)
6	Hardware	CPU is fine	GPU/TPU strongly preferred
7	Interpretability	High: feature importance, coefficients	Low: "black box" hidden layers

#	Aspect	Machine learning	Deep learning
—	Best for	Tabular/structured data, when explainability matters	Images, text, audio, video, very large or high-dimensional data

The turning point: in the 2012 ImageNet challenge (14M+ images, 20,000+ categories), the CNN **AlexNet**, trained on GPUs, cut top-5 error to about 15% vs. about 26% for the best non-deep approach. In 2015 Microsoft's ResNet (152 layers) went further. After that, industry moved to deep learning for perception tasks.

Typical DL use cases: image classification (X-rays, MRI, tumours), object detection (vehicles, helmets, number plates), face recognition, translation, summarisation, sentiment analysis, time-series forecasting, speech.

3. Deep learning model families

Family	Data it suits	What it does	Examples
MLP (multilayer perceptron; a.k.a. deep neural network, ANN)	Tabular data, flattened small images	Regression and classification	House prices, loan approval, MNIST digits
CNN / computer vision	Images (pixels 0–255; greyscale or RGB channels)	Classification, object detection (bounding boxes), segmentation	AlexNet, VGG, ResNet, MobileNet; YOLO for detection; medical imaging, number plates
RNN (recurrent)	Sequences where order matters: text, time series, audio, sensors	Next-step prediction, sentiment, forecasting	"I live in Bangalore, I speak ___"
LSTM (1997)	Longer sequences	RNN with **forget, input, output** gates to control memory	Forecasting, speech, text classification
GRU (2014)	Same as LSTM	Simpler LSTM with **update and reset** gates; faster, often similar accuracy	Same tasks as LSTM

Family	Data it suits	What it does	Examples
Encoder–decoder	Sequence in → sequence out	Translation, POS tagging, named-entity recognition	"I am teaching GenAI" → "Main GenAI padha raha hoon"
Transformer (2017)	Text first, now everything	Self-attention reads the whole context at once, in parallel	Foundation of all LLMs

Why order matters: "Hi, my name is Sundaram" makes sense; "My hi Sundaram is name" doesn't. RNNs process tokens in order but struggle to remember long contexts. LSTM/GRU help; Transformers solved it with attention.

Context matters (why transformers won): "*Apple* is good for health" (fruit) vs. "*Apple* plans to buy Samsung" (company). Attention uses the surrounding words to tell them apart, just like "bank" of a river vs. a bank.

BERT vs. GPT (both 2018, both Transformer-based)

	BERT	GPT
Architecture	Encoder-only, **bidirectional** (reads left and right)	Decoder-only, left-to-right
Training task	**Fill in the blank** (masked language modelling)	**Predict the next word**
Best at	Understanding: classification, NER, search	Generation: chat, writing, code
By	Google	OpenAI (founded 2015)

The paper to read: **"Attention Is All You Need"** (Vaswani et al., 2017), covered next.

4. How a neural network learns

Biological inspiration

Brain	Artificial network
Dendrites receive a signal (touching ice)	Inputs x_1, x_2, x_3 (features)

Brain	Artificial network
Cell body processes it	Weighted sum + bias
Axon fires (or doesn't)	Activation function
Axon terminal → "it's cold"	Output ŷ

Hubel and Wiesel's studies of the visual cortex (Nobel Prize, 1981) inspired CNNs.

One neuron, step by step

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \qquad \hat{y} = f(z)$$

1. **Weighted sum** (the lecture called it the "step function"): multiply inputs by weights and add the bias.
2. **Activation** f: decides the output (ReLU, sigmoid, tanh, softmax).

Perceptron (Rosenblatt, 1958): a single neuron with a threshold, e.g. if $x_1 + x_2 > 1$ then "yes" else "no". It can only separate data with a straight line, so it fails on problems like XOR. Stacking layers (**MLP**) fixes this.

Counting parameters (weights + biases)

For each layer: inputs × neurons weights + neurons biases.

Network	Calculation	Parameters
3 → 2 → 1	(3×2 + 2) + (2×1 + 1)	**11**
4 → 3 → 3 → 2 → 1	(12+3) + (9+3) + (6+2) + (2+1)	**38**
4 → 100 → 50 → 1	(400+100) + (5,000+50) + (50+1)	**5,601**
MNIST: 784 → 128 → 10	(100,352+128) + (1,280+10)	**101,770**

LLMs use the same arithmetic at a vastly larger scale (billions or trillions of parameters).

The training loop

1. **Forward pass** (feed-forward): inputs → layers → prediction ŷ.
2. **Loss:** how wrong the prediction is. Example: inputs 10, 20, 30 with all weights 1 → 60; actual 100 → error 40.
3. **Backpropagation:** use the chain rule to compute how much each weight contributed to the error.

4. **Optimizer update:** adjust weights to reduce the loss (e.g. weights 2, 1, 2 → 10×2 + 20×1 + 30×2 = 100 → loss 0).
5. Repeat for every **batch**; one full pass over the training data = one **epoch**.

The goal is loss close to zero (exactly zero usually means overfitting).

Key components

Component	Use	Notes
ReLU	Hidden layers (default)	Fast; avoids vanishing gradients. Try Leaky ReLU if neurons "die"
Sigmoid	Binary output	Range 0–1
tanh	Older hidden layers / RNNs	Range −1 to 1; prone to vanishing gradients
Softmax	Multi-class output	Probabilities over classes that sum to 1
Adam	Optimizer (default)	Others: SGD, RMSProp, AdaGrad, AdaDelta, Nadam
Loss	Regression: MSE/MAE. Binary: binary cross-entropy. Multi-class: categorical cross-entropy	
Dropout	Randomly switches off neurons during training	Reduces overfitting
Batch normalization	Normalises layer activations per batch	Faster, more stable training

Common training problems

Problem	What happens	Typical fixes
Vanishing gradient	Gradients shrink through layers; early layers stop learning; loss plateaus	ReLU, good initialisation, batch norm, residual connections, LSTM/GRU
Exploding gradient	Gradients blow up; loss swings wildly or becomes NaN	Gradient clipping, lower learning rate, normalisation
Local minima (non-convex loss)	Gets stuck in a poor minimum	Better optimizers (Adam, momentum)

The main failure modes: ML struggles with **bias and variance**; deep learning with **vanishing or exploding gradients**.

5. MLP on MNIST: code and results

Task: classify handwritten digits 0–9 (MNIST: 60,000 training and 10,000 test images, 28×28 greyscale). This is a **multi-class classification** problem with 10 classes.

The 6-step recipe

1. Import libraries
2. Load data
3. Preprocess: scale pixels to 0–1, flatten images, one-hot encode labels
4. Define the model (Sequential API)
5. Compile: optimizer, loss, metrics
6. Fit, then evaluate on train and test

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.datasets import mnist

# 2. Load
(X_train, y_train), (X_test, y_test) = mnist.load_data()
print(X_train.shape, X_test.shape)          # (60000, 28, 28) (10000, 28, 28)

# Look at 9 random digits
idx = np.random.randint(0, 60000, 9)
```

```
plt.figure(figsize=(6, 6))
for i, j in enumerate(idx):
    plt.subplot(3, 3, i + 1)
    plt.imshow(X_train[j], cmap="gray")
    plt.title(y_train[j])
    plt.axis("off")
plt.show()

# 3. Preprocess
X_train = X_train.reshape(60000, 784).astype("float32") / 255.0   # flatten
28x28 -> 784, scale 0-1
X_test  = X_test.reshape(10000, 784).astype("float32") / 255.0
y_train_cat = keras.utils.to_categorical(y_train, 10)            # one-hot
labels
y_test_cat  = keras.utils.to_categorical(y_test, 10)

# 4. Define
model = Sequential([
    Dense(128, activation="relu", input_shape=(784,)),   # hidden layer
    Dense(10, activation="softmax"),                       # 10 classes
])
model.summary()                                             # 101,770
trainable parameters

# 5. Compile
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])

# 6. Fit + evaluate
history = model.fit(X_train, y_train_cat,
                    validation_data=(X_test, y_test_cat),
                    epochs=10, batch_size=128)
model.evaluate(X_test, y_test_cat)
```

The class error ("incompatible input shape") happened because 28×28 images were fed into `Dense(..., input_dim=784)`. The fix is to **reshape to 784** (or add a `Flatten(input_shape=(28, 28))` layer first).

Alternative: keep integer labels and use `loss="sparse_categorical_crossentropy"`, which skips one-hot encoding.

Results from class

Setup	Train accuracy	Test accuracy	Loss
ReLU + Adam + batch 128 (epoch 1 → 10)	83% → 99%	94% → 97–98%	falls to ~0.02 (train), ~0.07 (test)
tanh + AdaGrad, default batch	~90–91% (plateaus)	~91%	stuck ~0.29–0.30

The second run shows slow, stalled learning, the symptom the lecture used to illustrate vanishing gradients. On harder RGB data (CIFAR-10/100), an MLP does much worse; that's why CNNs exist.

Choosing hidden neurons: powers of two (32, 64, 128, 256, 512) are common starting points. Tune by validation accuracy, or automatically with tools like KerasTuner.

6. Q&A highlights

Question	Answer
Why deep learning before memory/tools?	Transformers and fine-tuning rest on these ideas (weights, optimization, embeddings). Agents and tools come after fine-tuning.
How much DL coding do interviews need?	Very little for GenAI roles. Know the concepts and Python basics; DL coding only matters for client projects using classic DL.
How does DL relate to embeddings in RAG?	Embeddings are produced by Transformer networks, so understanding DL explains how text becomes numbers.
Can an RNN predict the next word?	Yes, but it forgets earlier context quickly. LSTM/GRU remember more; Transformers handle long context far better.
Does BERT have a length limit?	Yes: 512 tokens. Modern LLMs handle much longer contexts, and RAG plus chunking covers the rest.

Question	Answer
How did models get more parameters?	More layers and neurons, more data, more compute (GPUs). Bigger isn't always better: e.g. ResNet-50 (~25M parameters) beat VGG-16 (~138M).
What is "long context"?	A model's ability to accept very long inputs (e.g. whole documents in one prompt). Tools like Gamma use LLMs to *generate* long content (whole slide decks) from a short prompt; that's long-form generation.
Is a bias always added per neuron?	Yes, one bias per neuron. It works like an intercept: e.g. a fresher with zero experience still gets a base salary.
Why did the perceptron fail?	A single neuron can only draw a linear boundary. Real problems are non-linear, and stacking layers (MLP) plus good optimizers handles that.
How many hidden neurons should I use?	Start with powers of two and tune on validation data, or use automated search.
Can Keras/TensorFlow build Transformers?	Yes, but most people use Hugging Face `transformers` (on PyTorch or TensorFlow).
Schedule	Saturday 10:15–10:30 start, Sunday 10:00 IST, ending 13:30. Project batch evenings 7–9 PM.

7. Expert corrections to the lecture

Concepts

1. **BERT and GPT tasks were swapped on the board.** BERT (encoder) = fill in the blank (masked LM); GPT (decoder) = next-word prediction. BERT came from Google AI Language, not DeepMind.
2. **Computer vision isn't "unsupervised".** Object detection (bounding boxes) is supervised: it learns from labelled boxes. Anomaly detection is a different task.
3. **The perceptron failed on non-linear problems.** A single neuron can't solve XOR (Minsky & Papert, 1969). That limit, not "convex vs non-convex layers", contributed to the first AI winter.
4. **The class's parameter count for 4 → 100 → 50 → 1 was garbled.** It is 5,601 (see Section 4).

5. **An epoch is one full pass over the training data,** made of many batches, each with its own forward pass, backprop and weight update. It's not "one backpropagation".
6. **Removing `batch_size` doesn't send all 60,000 images at once.** Keras defaults to batches of 32.
7. **Dropout reduces overfitting** by randomly disabling neurons during training. It isn't for speed.
8. **Batch normalization isn't min-max scaling.** It standardises each layer's activations per batch (mean 0, variance 1), with learnable scale and shift.
9. **Keras API choice is about topology, not model type.** The Functional API handles non-linear topologies (multiple inputs/outputs, skip connections); Sequential suits simple stacks. RNNs can use either.
10. **The MNIST error needed flattening.** It was fixed by reshaping to 784 (or a `Flatten` layer); the lecture said flatten wasn't needed for an MLP.
11. **Vanishing gradient** = gradients shrink so early layers stop learning (loss plateaus). **Exploding** = gradients grow so training becomes unstable (loss jumps or NaN). Both are classic RNN problems, not "MLP vs RNN" respectively.
12. **Activation ranges:** sigmoid 0–1, tanh −1 to 1.
13. **LSTM has no fixed "30-word" limit.** It degrades gradually over long sequences.
14. **Long-context windows** (long inputs) aren't the same as long-form generation (Gamma's slide decks).
15. **Deep models aren't automatically better than ARIMA for time series.** On small or simple series, classic models often win.

History

16. **Perceptron:** Frank Rosenblatt (an American psychologist), 1958, not a Russian mathematician. **RNNs** date from the 1980s (Hopfield 1982, Elman 1990), not 1970.
17. **Backpropagation** was popularised in 1986 (Rumelhart, Hinton & Williams). Hinton's 2006 deep-belief-net work revived deep learning. He worked at Google 2013–2023 and shared the **2024 Nobel Prize in Physics** with John Hopfield.
18. **Dropout:** Srivastava, Hinton et al. (2012 preprint, 2014 JMLR paper).
19. **AlexNet** (Krizhevsky, Sutskever, Hinton) was a breakthrough, not a "poor model": ~15.3% vs ~26.2% top-5 error.
20. **AI winters** were roughly 1974–80 and 1987–93.
21. **"Attention Is All You Need"** was published in June 2017 (NeurIPS, Dec 2017).
22. **OpenAI funding:** Musk left OpenAI's board in 2018; Microsoft invested $1B in 2019; Musk bought Twitter in 2022. The lecture blended these.
23. **Tools:** PyTorch is from Meta (2016); Keras became TensorFlow's official API in TF 2.0 (2019). The library is Theano (not "Tiano").

8. Interview quick revision

> **ML vs. DL?** ML relies on hand-engineered features and works well on smaller, tabular data with high interpretability. DL learns features automatically through many hidden layers, needs more data and GPUs, is less interpretable, and dominates images, text, audio and other unstructured data.

> **How does a neural network learn?** Forward pass (weighted sums + activations) → loss → backpropagation (chain rule) computes gradients → an optimizer such as Adam updates the weights → repeat over batches and epochs until the validation loss stops improving.

> **MLP vs. CNN vs. RNN vs. Transformer?** MLP for tabular data. CNN for spatial data (images). RNN/LSTM/GRU for sequences, though they struggle with long context. Transformers use self-attention to model long context in parallel, the basis of BERT (encoder, understanding) and GPT (decoder, generation).

> **Vanishing vs. exploding gradients?** Vanishing: gradients shrink, so learning stalls (fix: ReLU, normalisation, residuals, LSTM). Exploding: gradients blow up, so training goes unstable (fix: gradient clipping, lower learning rate).

Practice

- [ ] Re-run the MNIST MLP; try 64/256 neurons, add Dropout, change epochs
- [ ] Try the same MLP on CIFAR-10 (RGB) and see why CNNs are needed
- [ ] Read "Attention Is All You Need" before the transformer session

Next: CNN and LSTM use cases, then Transformers in depth, then fine-tuning.

SESSION 10

Sunday, 5 July 2026

CNNs, Transfer Learning & RNNs

Deep Learning Notes – Session 10 (5 July 2026)

Oct 14, 2026 · @Vijay

1. Convolutional neural networks (CNNs)

A CNN is a neural network built for images. It learns **edges → corners → shapes → objects** layer by layer, exploiting the spatial structure of pixels. In practice, start from a **pre-trained CNN** (transfer learning) instead of training from scratch.

Use cases: image classification, object detection, face recognition (phone face unlock), medical image analysis, document and handwriting processing.

Why not an MLP for images?

	MLP	CNN
Works well for	Tabular data, small greyscale images (MNIST 28×28)	Large and/or colour (RGB, 3-channel) images
Input	Flattened pixels: 256×256 → 65,536 inputs, which explodes the parameter count	Local filters shared across the image, so far fewer parameters
Spatial awareness	None: neighbouring pixels are treated as unrelated	Built in: filters scan local patches

Building blocks

Layer / setting	What it does
`Conv2D(filters, kernel_size)`	Slides small learnable filters (e.g. 32 filters of 3×3) over the image; each produces a feature map (edges, textures…)
Kernel size	Filter window, e.g. 3×3 (AlexNet's first layer used 11×11)
Stride	How far the filter moves each step: stride 2 halves the output size

Layer / setting	What it does
Padding	`valid` = no padding, so output shrinks; `same` = zero-pad so output keeps its size
`MaxPooling2D(2×2)`	Keeps the largest value in each window, which downsamples, cuts computation and adds slight shift tolerance (some detail is lost)
`BatchNormalization`	Normalises activations, giving faster and more stable training
`Dropout`	Randomly disables neurons during training to reduce overfitting
L2 regularizer (ridge)	Penalises large weights to reduce overfitting
`Flatten`	Turns the final feature maps into a 1-D vector
`Dense` layers	MLP head that does the final classification

CNN = convolution/pooling feature extractor + MLP classifier. That's why the MLP comes first in the curriculum.

2. CNN code: cats vs dogs, then transfer learning

Task: classify RGB images as cat or dog (Kaggle "Dogs vs Cats"). The live download failed in class (Kaggle API 403), so the code was explained line by line.

Getting Kaggle data into Colab

```
!mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600
~/.kaggle/kaggle.json
!kaggle datasets download -d salader/dogs-vs-cats
import zipfile
zipfile.ZipFile("dogs-vs-cats.zip").extractall("/content")
```

A 403 usually means the dataset's rules haven't been accepted on the Kaggle website, or `kaggle.json` is in the wrong place or has the wrong permissions.

Model built from scratch

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten,
```

```
BatchNormalization, Dropout

train_ds = keras.utils.image_dataset_from_directory(
    "/content/train", labels="inferred", label_mode="int",   # cat=0, dog=1
(alphabetical)
    batch_size=32, image_size=(256, 256))
val_ds = keras.utils.image_dataset_from_directory(
    "/content/test", labels="inferred", label_mode="int",
    batch_size=32, image_size=(256, 256))

normalize = lambda img, label: (tf.cast(img / 255.0, tf.float32), label)   #
scale 0-1
train_ds, val_ds = train_ds.map(normalize), val_ds.map(normalize)

model = Sequential([
    Conv2D(32, (3, 3), padding="valid", activation="relu", input_shape=(256,
256, 3)),
    BatchNormalization(), MaxPooling2D((2, 2), strides=2, padding="valid"),
    Conv2D(64, (3, 3), padding="valid", activation="relu"),
    BatchNormalization(), MaxPooling2D((2, 2), strides=2, padding="valid"),
    Conv2D(128, (3, 3), padding="valid", activation="relu"),
    BatchNormalization(), MaxPooling2D((2, 2), strides=2, padding="valid"),
    Flatten(),
    Dense(128, activation="relu"), Dropout(0.1),
    Dense(64, activation="relu"), Dropout(0.1),
    Dense(1, activation="sigmoid"),          # binary: cat vs dog
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])
history = model.fit(train_ds, epochs=10, validation_data=val_ds)
```

- Filters double each block (32 → 64 → 128), the same pattern VGG uses.
- Predict a new image by resizing it to 256×256, scaling it, and calling `model.predict`. A result ⩾ 0.5 means dog (1); below 0.5 means cat (0).
- For more than 2 classes, use `Dense(n, activation="softmax")` with categorical cross-entropy.

Transfer learning (the recommended way)

Transfer learning = reuse a model pre-trained on a huge dataset (e.g. ImageNet) and adapt it to your task. It's the same idea as fine-tuning an LLM.

Pre-trained model	Year	Notes
AlexNet	2012	Input 227×227×3, first layer 96 filters of 11×11 with stride 4; ~60M parameters; ~15% top-5 error
VGG-16	2014	Input 224×224×3; 13 conv + 3 dense layers in 5 blocks; ~138M parameters
ResNet-50	2015	Residual connections; ~25M parameters
Xception, MobileNet	2016–17	Efficient; MobileNet is very small (~4M)

These are all available in `keras.applications` with ImageNet weights.

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, Model

base = VGG16(weights="imagenet", include_top=False, input_shape=(224, 224,
3))
base.trainable = False                        # 1) freeze the convolutional
base

x = layers.GlobalAveragePooling2D()(base.output)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)
out = layers.Dense(10, activation="softmax")(x)
model = Model(base.input, out)
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=
["accuracy"])
# 2) train the new head, then optionally unfreeze the last block (block5)
with a small learning rate
```

Fine-tuning strategies:

- Data similar to ImageNet, or a small dataset → **freeze the whole base** and train only the new head.
- Data quite different, or a large dataset → also **unfreeze the top block(s)** (e.g. VGG block5) and train with a low learning rate.
- `include_top=False` drops the original 1,000-class head so you can attach your own.

3. Data augmentation and translation invariance

Data augmentation creates many altered copies of each training image (shifted, rotated, zoomed, flipped, brightened), so the model recognises objects regardless of position or viewpoint. It also multiplies a small dataset.

Term	Meaning
Translation **equivariance**	Shift the input and the feature map shifts the same way (a property of convolution)
Translation **invariance**	The output label stays the same when the object moves (helped by pooling and augmentation)
Rotation/scale robustness	Not built into CNNs; learned through augmentation

Why it matters: a self-driving car must recognise a pedestrian whether centred, at the edge, rotated or partly in shadow.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# or the newer layers: keras.layers.RandomFlip / RandomRotation / RandomZoom

datagen = ImageDataGenerator(
    rotation_range=20, width_shift_range=0.2, height_shift_range=0.2,
    shear_range=0.2, zoom_range=0.2, horizontal_flip=True,
    brightness_range=(0.8, 1.2), fill_mode="nearest")
```

Augmentation doesn't tell real images from fake ones; that's a job for **GANs** (generative adversarial networks).

4. Using GenAI to write deep-learning code

You don't need to memorise DL code. You need to understand it well enough to prompt for it, read it and debug it.

Prompt used in class:

```
Act as an expert in generating deep learning code in Python.
Your task is to provide complete end-to-end code for a CNN problem.
Dataset: CIFAR-10 small image classification (Keras datasets).
Use the VGG-16 pre-trained model (transfer learning).
Ensure accuracy above 90%.
Add headers explaining what each block of code does.
Use data augmentation as well.
```

The generated code resized images to 224×224×3 for VGG-16, one-hot encoded 10 classes, used `ImageDataGenerator`, froze the VGG base (`include_top=False`, `trainable=False`) and added a GlobalAveragePooling + Dense + Dropout head. It trained the head first, then unfroze **block5** for fine-tuning, and plotted accuracy and loss curves.

Because you understand each concept, you can steer the output: "use MaxPooling instead of GlobalAveragePooling", "unfreeze one more block", "lower the learning rate".

5. RNN basics

RNNs handle sequences of variable length where order matters (text, time series). An MLP or CNN needs a fixed-size input; sentences have different word counts (5, 6, 7, 9 words…). For new NLP work, Transformers have replaced RNNs; learn RNNs for background.

Step 1 – Tokenise (text → integer IDs)

```
from tensorflow.keras.preprocessing.text import Tokenizer

docs = ["Hi my name is Sundaram",
        "GenAI is really good nowadays",
        "You can build a RAG system",
        "Agentic AI is another skill in market",
        "MCP is a protocol in LLM",
        "A2A is something which communicates with agent to agent"]

tok = Tokenizer(oov_token="<OOV>")   # unseen words map to index 1
tok.fit_on_texts(docs)
tok.word_index       # {'<OOV>': 1, 'is': 2, 'a': 3, ...}  32 entries
tok.word_counts      # e.g. 'is' appears 4 times
tok.document_count   # 6 (each sentence = one document)
seqs = tok.texts_to_sequences(docs)   # "Hi my name is Sundaram" -> [5, 6, 7,
2, 8]
```

This is **encoding**: assigning IDs, with no meaning attached.

Step 2 – Pad to equal length

```
from tensorflow.keras.utils import pad_sequences
padded = pad_sequences(seqs, padding="post")   # zeros added at the end, up
to the longest (9)
# padding="pre" adds zeros at the start instead
```

Step 3 – SimpleRNN model

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(32, input_shape=(50, 1)),     # 50 time steps, 1 feature
    Dense(1, activation="sigmoid"),
])
model.summary()
```

Parameter count (1,121):

- SimpleRNN: input weights 1×32 + **recurrent weights 32×32** + biases 32 = 1,088
- Dense: 32×1 + 1 = 33

The 32×32 recurrent matrix is what makes it "recurrent": each step's hidden state feeds into the next step.

RNN → LSTM → GRU

Model	Fix	Gates
RNN	—	Forgets long-range context (vanishing gradients)
LSTM	Adds a memory cell	Forget, input, output
GRU	Simpler, faster LSTM	Update, reset

In practice, use Transformer-based models for text. For RNN/LSTM internals, a short Medium explainer (about 4 minutes) is enough.

6. Interview answer: "How do you debug code?"

Debugging is a structured process, not a Python trick. (A student was asked this in an interview.)

1. **Define the failure:** expected output vs actual output (or the error raised).
2. **Reproduce it consistently**, then run step by step or line by line, not all at once.
3. **Read the logs and stack traces:** application, error, server, API, database and cloud-monitoring logs (observability tools in production).
4. **Classify the error:** syntax, timeout, authentication, missing dependency, network, data/format.

5. **Isolate the layer:** UI → API → business logic → data → external services. Test each separately.
6. **Check the data and external dependencies.** Many production bugs are data issues, e.g. the client sent XML instead of the agreed JSON, or a cloud service or auth configuration changed.
7. **Fix one thing at a time**, re-test, and add a test or log line so it doesn't recur.

> "I follow a structured approach: understand expected vs actual behaviour, reproduce the issue, read the logs and traces to locate the failing layer, isolate it, check data and dependencies, then fix and verify one change at a time."

7. Q&A highlights

Question	Answer
MLP vs CNN – both use pixels?	MLP is fine for small greyscale images or tabular data. For RGB or large images, use a CNN; an MLP performs badly there.
How do I choose the train/test split?	Common splits are 70/30, 75/25 or 80/20. `train_test_split` defaults to 75/25. Keras datasets like CIFAR come pre-split (50k/10k).
Why 32 filters and a 3×3 kernel?	Hyperparameters. Common practice copies proven architectures (VGG-style 32 → 64 → 128).
Is there Conv3D for video?	Yes, `Conv3D` exists. Video is also handled with CNN + temporal models or video transformers.
Which layers do I freeze when fine-tuning?	See Section 2: usually freeze the base and train the head, then unfreeze the top block if needed.
Is DL coding asked in GenAI interviews?	Rarely. Concepts matter; GenAI can write the code.
Do I need DSA / advanced Python?	Only for software-engineering roles. For GenAI roles, basic Python is enough.

Question	Answer
Encoding vs embedding?	Encoding maps tokens to IDs or one-hot vectors (sparse, no meaning). Embeddings are learned dense vectors that capture meaning and context, stored in vector DBs for RAG.
Can I retrain GPT on my newer private data?	Training a foundation model is too costly. Use RAG, fine-tune an open model, or expose live data through MCP/tools.
How do chatbots answer about recent events?	Through web-search tools used by the agent layer. The model's knowledge itself still stops at its training cutoff.
Private cloud and fine-tuning vs RAG in enterprises?	Many firms do both: RAG for most data; fine-tuned or self-hosted open models in private clouds for highly regulated data. Cloud platforms (Azure, AWS, GCP) are the norm.
What is Google Antigravity?	An agent-first IDE where AI agents plan, write, test and debug code with minimal human input (like Windsurf or Cursor).
How do I know my chunking/embedding is good?	Evaluate retrieval with context precision, recall and relevancy against a threshold, then iterate (covered later).
Book recommendation?	"Hands-On Large Language Models" (Jay Alammar & Maarten Grootendorst); free notebooks on GitHub. The lecturer found it underwhelming.
What's next?	Transformers (11–12 July), fine-tuning (PEFT, LoRA, QLoRA), then tool/function calling, memory, MCP/A2A and harnesses (~18–19 July onward).

8. Expert corrections to the lecture

1. **The opening advice was garbled.** It means: *don't* build CNNs from scratch, *do* use pre-trained models via **transfer learning** (the lecture also said "transformer learning" by mistake).

2. **Max pooling** keeps the maximum of each window and outputs a smaller map. It doesn't copy the max to every cell. It has no parameters itself but shrinks everything downstream.
3. **Padding is not about making images the same size.** `valid` = no padding (output shrinks); `same` = zero-padding to keep size. Equal image sizes come from **resizing** (`image_size=(256, 256)`).
4. **Translation equivariance vs invariance** is a property of CNN feature maps (see Section 3). It's unrelated to language translation; the lecture's "Mai ghar ja raha hoon" analogy mixed the two up.
5. **Filter counts (32/64/128) are hyperparameters** inspired by VGG-style designs. They weren't "taken from AlexNet".
6. **Keras doesn't auto-split 75/25.** `model.fit` has `validation_split=0.0` by default. 75/25 is scikit-learn's `train_test_split` default.
7. **2-D convolution isn't the only option.** `Conv3D` exists for video and volumetric data.
8. **`include_top=False`** removes the classifier head; `trainable=False` is what freezes weights.
9. **The SimpleRNN parameter count is 1,121** (1,088 + 33). The recurrent connections are a 32×32 weight matrix across time steps, not "connecting back 32 times".
10. **Tokenizer OOV token:** use `oov_token="<OOV>"`. The "abcde…" placeholder works, but it's just a label for unknown words.
11. **"Never use RNN/LSTM" is too strong.** They're legacy for NLP but still used for some time-series, streaming and edge cases. RNNs/LSTMs dominated NLP roughly 2014–2017.
12. **CAG isn't "already covered"** and isn't knowledge-graph RAG. It usually means **Cache-Augmented Generation**: preloading a small, stable knowledge base into the model's context/KV cache instead of retrieving per query.
13. **Perplexity-style freshness comes from live web retrieval**, not specifically MCP. MCP is one standard for connecting tools and data.
14. **Dropout isn't bootstrapping.** The random-forest analogy is loose: dropout randomly zeroes activations during training as regularisation.
15. **AlexNet** was by Krizhevsky, Sutskever and Hinton (Hinton's students), published in 2012.

9. Quick revision and what's next

> **CNN in one breath:** convolution layers with small shared filters detect edges, then shapes, then objects. Pooling downsamples. Batch norm and dropout stabilise and regularise. Flatten feeds an MLP head with sigmoid (binary) or softmax (multi-class). In practice, fine-tune a pre-trained model (VGG/ResNet/MobileNet) with augmentation.

> **Transfer learning:** load ImageNet weights with `include_top=False`, freeze the base, add your own head, train, then optionally unfreeze the top block with a low learning rate.

> **RNN in one breath:** tokenise text to IDs, pad to equal length, process step by step with a recurrent hidden state. Vanishing gradients limit memory, LSTM/GRU add gates, and Transformers replaced them all for NLP.

Practice

- [ ] Run the GenAI-generated VGG-16 CIFAR-10 notebook on a Colab GPU
- [ ] Fix the Kaggle setup (accept the dataset rules, `chmod 600`) and train the cats-vs-dogs CNN
- [ ] Read "Attention Is All You Need" before next session

Next session: the Transformer paper end to end (embeddings, positional encoding, self- and multi-head attention, encoder and decoder, BERT and GPT), then fine-tuning.

SESSION 11

Saturday, 11 July 2026

Transformers & BERT

Transformer & BERT Notes – Session 11 (11 July 2026)

Oct 14, 2026 · @Vijay

1. Why transformers

"Attention Is All You Need" (Vaswani et al., Google, 2017) introduced the Transformer. It replaced RNNs and LSTMs with attention alone, and it underpins GPT, BERT, Gemini, Claude, Llama, Copilot, RAG and agentic AI. Expect interview questions on it.

The problem with RNNs and LSTMs

	RNN	LSTM
How it reads	Word by word, in order ("I → love → learning → AI")	Same, with gates to keep memory longer
Memory	Forgets quickly	Forgets gradually; still struggles with long text
Speed	Slow: each step waits for the previous one	Slow for the same reason
Scaling	Hard to train at large scale	Hard to train at large scale

Example: "The animal didn't cross the road because **it** was tired." The model must link "it" to "animal", which is hard when context is held only in a step-by-step memory.

What the Transformer introduced

1. **Attention:** every word looks directly at every other word to build its meaning.
2. **Parallel processing:** all tokens are processed at once, not one after another.
3. **Better context understanding:** long-range relationships are captured directly.
4. **Faster training and better accuracy:** parallelism suits GPUs, so models can scale to billions of parameters.

> RNN/LSTM: W1 → W2 → W3 → W4 (sequential). Transformer: W1, W2, W3, W4 all at once (parallel).

2. The road to attention

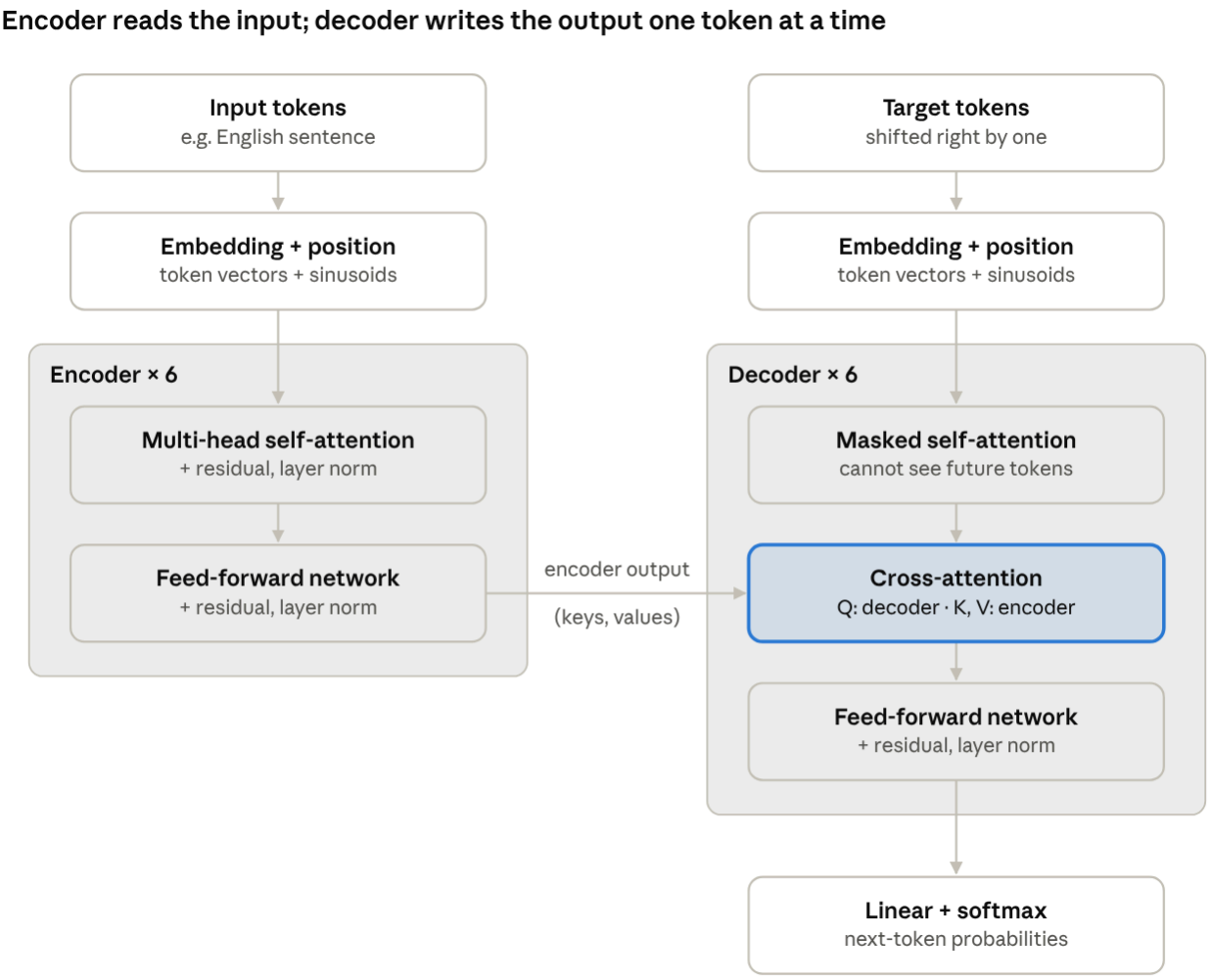
Year	Paper	Idea	Limitation
2014	"Sequence to Sequence Learning with Neural Networks" (Sutskever, Vinyals, Le; Google)	**Encoder–decoder:** an RNN/LSTM encoder squeezes the whole input into one fixed **context vector**; a decoder generates the output from it	Fixed-size vector loses information on long inputs; still sequential RNNs
2015	"Neural Machine Translation by Jointly Learning to Align and Translate" (Bahdanau, Cho, Bengio)	**Attention:** at each output step, the decoder looks back at *all* encoder states and weights the relevant ones	Still built on RNNs, so still sequential and slow
2017	"Attention Is All You Need" (Vaswani et al.; Google)	**Drop recurrence entirely:** self-attention + feed-forward layers, fully parallel	Large compute and data needs

Analogy for seq2seq: read a 1,000-page book, then answer questions on any page from memory. Information gets lost. Attention lets you flip back to the relevant pages.

Seq2seq was good for translation, summarisation and Q&A, and established the encoder–decoder pattern the Transformer kept.

3. Transformer architecture

The original Transformer is an encoder stack plus a decoder stack, each 6 layers deep. The cross-attention link (highlighted) is where the decoder reads what the encoder understood.



Transformer encoder–decoder · base model, N = 6

Encoder (understands the input)

1. **Input embedding:** tokens become vectors (computers only understand numbers; each token gets its own vector "ID card").
2. **Positional encoding:** sine/cosine position signals are added, because attention by itself ignores order ("dog bites man" ≠ "man bites dog").
3. **Multi-head self-attention:** every token attends to every other token.
4. **Add & Norm:** residual connection + layer normalisation.
5. **Feed-forward network:** a small MLP applied to each token separately, then Add & Norm again.
6. Repeat N = 6 times. The final output is the encoder representation.

Decoder (generates the output)

1. **Output embedding + positional encoding** of the target tokens, shifted right.
2. **Masked multi-head self-attention:** each position sees only earlier tokens.

3. **Cross-attention:** queries from the decoder; keys and values from the encoder output.
4. **Feed-forward network** (each sub-layer followed by Add & Norm). Repeat N = 6 times.
5. **Linear + softmax** gives a probability for every vocabulary word; pick the next token and repeat.

	Encoder-only	Decoder-only	Encoder–decoder
Example	BERT	GPT, Llama, Claude	Original Transformer, T5
Good at	Understanding (classification, NER, search)	Generation (chat, writing, code)	Sequence-to-sequence (translation, summarisation)

Analogy: Alexa first **understands** your question (NLU, encoder-like), then **generates** an answer (NLG, decoder-like).

4. Self-attention: query, key, value

Self-attention lets each token build a context-aware representation by weighting every other token in the sentence. It's how "bank" in "river bank" ends up different from "bank" in "bank loan", and how "it" links to "animal".

Q, K, V: the library analogy (a top interview question)

	Meaning	Library analogy
Query (Q)	What this token is looking for	"I'm searching for machine learning"
Key (K)	What each token offers to be matched against	Book titles: "Machine Learning Fundamentals"
Value (V)	The information actually passed on once a match is found	The book's contents

Each token's embedding is multiplied by three learned weight matrices (W_Q, W_K, W_V) to produce its Q, K and V vectors. These weights are learned during training by backpropagation.

The formula (scaled dot-product attention)

```
\text{Attention}(Q,K,V) = \text{softmax}\!\left(\frac{QK^{\top}}
{\sqrt{d_k}}\right)V
```

1. **Score:** dot product of a token's query with every key. A higher score means more relevant.
2. **Scale:** divide by √d_k (d_k = 64, so divide by 8). Large dot products would push softmax into extreme values with tiny gradients; scaling keeps training stable.
3. **Softmax:** turn the scores into weights that sum to 1 (e.g. for "love" in "I love AI": I = 35%, AI = 65%).
4. **Weighted sum of values** gives the token's new **contextual embedding**.

Example from class: in "Python execute quickly", the query for "Python" attends most to itself, then "execute", then "quickly". The same happens in parallel for every token. In "He got bit by a python" vs "Python is my favourite programming language", attention gives the two "python"s different meanings.

Why separate Q, K and V?

Using raw embeddings directly would give one fixed way of relating tokens, and "bank" would always mean the same thing. Separate learned projections let the model learn *different kinds of relationships* (syntactic, semantic), so each token's representation adapts to context.

Multi-head attention

The Transformer runs **8 attention heads in parallel** (BERT-base: 12). Each head has its own Q/K/V projections and can focus on a different pattern: grammar, coreference ("it" → "animal"), topic. Their outputs are concatenated and projected back. It's like several readers each looking at the sentence from a different angle.

5. BERT

BERT (Bidirectional Encoder Representations from Transformers, Google, 2018) is an encoder-only Transformer. It reads context on both sides of every word at once, producing context-based embeddings for understanding tasks. In practice you take pre-trained BERT and **fine-tune** it (transfer learning) rather than training from scratch. GPT is the decoder-only counterpart, built for generation.

Configurations

	Original Transformer (base)	BERT-base	BERT-large
Layers	6 encoder + 6 decoder	12 encoder	24 encoder
Attention heads	8	12	16
Hidden size	512	768	1,024
Parameters	~65M	~110M	~340M

Cased vs. uncased: *uncased* lower-cases all text before tokenising; *cased* keeps capitalisation (useful when case carries meaning, e.g. names or "US" vs "us").

Input representation (three embeddings added together)

Example input: "I am learning GenAI" + "BERT is powerful" becomes `[CLS] I am learning GenAI [SEP] BERT is powerful [SEP]`.

Embedding	Encodes
Token	The WordPiece token itself
Segment	Which sentence (A or B) the token belongs to
Position	The token's position in the sequence (max 512)

`[CLS]` starts the sequence; its final vector is used for classification. `[SEP]` separates or ends sentences.

WordPiece tokenizer

BERT's vocabulary has ~30,000 sub-word pieces. An unknown word is split into known pieces, marked with `##`: "pretraining" → `pre`, `##train`, `##ing`. This handles out-of-vocabulary words, which Word2Vec, GloVe and FastText-era models struggled with.

Pre-training tasks

1. **Masked Language Modelling (MLM):** pick 15% of tokens. Of those, 80% become `[MASK]`, 10% are swapped for a random token, 10% are left unchanged. The model predicts the originals using both left and right context.
2. **Next Sentence Prediction (NSP):** given sentences A and B, predict whether B really follows A.

Training data: BooksCorpus (~800M words) + English Wikipedia (~2.5B words).
Activation: **GELU** (Gaussian Error Linear Unit), a smoother alternative to ReLU, also used in GPT.

6. Evaluation metrics for language models

Metric	Measures	Typical use
Perplexity	How "surprised" a model is by real text; lower is better	Language modelling
BLEU	n-gram precision against reference text	Machine translation
ROUGE	n-gram/sequence recall against reference summaries	Summarisation
METEOR	Matches including stems and synonyms	Translation
GLUE	Benchmark *suite* of 9 understanding tasks	Comparing NLU models (BERT-era)

GLUE tasks: CoLA (grammatical acceptability), SST-2 (sentiment), MRPC (paraphrase), STS-B (sentence similarity), QQP (duplicate questions), MNLI (entailment), QNLI (does a sentence answer a question), RTE (entailment), WNLI (pronoun resolution).

Don't confuse **GLUE** (a benchmark) with **GELU** (an activation function).

RAG and LLM-specific evaluation (faithfulness, relevancy, LLM-as-judge) comes later.

7. Q&A highlights

Question	Answer
What is "masked" multi-head attention?	In the decoder, each position may attend only to earlier positions (future tokens are masked), so the model can't peek at the word it's meant to predict.
Why is the decoder input "shifted right"?	During training the decoder receives the target sequence offset by one (starting with a start token), so at each step it predicts the next token from the previous ones.

Question	Answer
Does the decoder use the encoder's output?	Yes, through cross-attention: queries come from the decoder, keys and values from the encoder output.
Which embeddings did the original paper use?	Learned token embeddings over a sub-word vocabulary, plus sinusoidal positional encodings. Word2Vec/GloVe were earlier, separate techniques.
What does N× mean in the diagram?	The block is stacked N times: 6 encoder layers and 6 decoder layers in the base model.
Why the feed-forward network?	A position-wise two-layer MLP that further transforms each token's representation after attention.
Why divide by √d_k with d_k = 64?	d_k = d_model / heads = 512 / 8 = 64. Scaling by √64 = 8 keeps dot products in a range where softmax gives useful gradients.
Are Q, K, V learned?	Yes. The projection matrices are trained by backpropagation along with the rest of the model.
What is multi-head attention in practice?	The same attention computed several times in parallel with different learned projections; outputs are concatenated.
Can I use the lecturer's handwritten notes?	Notes and PPT are uploaded to the class folder. Make your own notes too.
Career advice	Start applying for GenAI/agentic roles now; remaining topics (tool calling, MCP, evaluation) are 2–3 weeks away.

8. Expert corrections to the lecture

1. **The decoder *does* connect to the encoder.** Cross-attention takes keys and values from the encoder output. The lecture said there was "no connection".
2. **Masked attention is causal masking,** not "hiding words to test the model". Random word hiding is BERT's MLM pre-training trick, a different thing.
3. **The 6 layers are stacked sequentially** (each feeds the next). Parallelism is across tokens and across heads, not across layers.

4. **The feed-forward sub-layer isn't there to compute loss or do backprop.** It's a per-token MLP. Loss is computed once at the output.
5. **"Add & Norm" = residual connection + layer normalisation,** not batch normalisation.
6. **Positional encodings are sinusoidal vectors added to embeddings,** not plain indices 1, 2, 3.
7. **Base-model sizes:** d_model = 512 (not "524"), feed-forward size 2,048, 8 heads, d_k = 64. The 64 follows from 512/8; it wasn't chosen because "8 is 2^3".
8. **BERT is bidirectional because self-attention sees both sides at once.** It doesn't read left-to-right and then right-to-left; that describes ELMo.
9. **MLM 80/10/10 rule:** of the 15% selected tokens, 80% → `[MASK]`, 10% → random token, 10% unchanged. The lecture's "100 masked + 25/25" split was garbled.
10. **BERT sizes are ~110M and ~340M parameters** (million, not billion). Pre-training data was BooksCorpus + English Wikipedia, not "Google blogs".
11. **"Google developed ChatGPT (encoder)" was a slip.** Google built BERT (encoder); OpenAI built GPT/ChatGPT (decoder).
12. **Decoder-only isn't "better" than encoder-only.** They suit different jobs: generation vs. understanding/classification.
13. **GLUE is a benchmark suite,** not an optimization method. Its tasks are CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, WNLI.
14. **Bahdanau attention (2015)** used a bidirectional RNN encoder with additive attention; co-authors were Cho and Bengio.
15. **GloVe was published in 2014.** LSTMs have no fixed "30-word" limit.

9. Interview quick revision

> **Why did Transformers replace RNNs?** RNNs process tokens sequentially, so they're slow to train and forget long-range context. Transformers use self-attention to relate every token to every other token in parallel, capturing long dependencies and scaling to huge datasets on GPUs.

> **Explain self-attention.** Each token is projected into a query, key and value. Attention scores are $\text{softmax}(QK^T/\sqrt{d_k})$; the output is the weighted sum of values, giving a context-aware embedding. Multi-head attention runs this several times in parallel to capture different relationships.

> **Encoder vs. decoder?** The encoder (BERT) builds bidirectional representations for understanding tasks. The decoder (GPT) uses masked self-attention to generate text left to right. In translation-style Transformers, the decoder also cross-attends to the encoder output.

> **How was BERT trained?** Masked language modelling (15% of tokens; 80/10/10) plus next-sentence prediction, on BooksCorpus and Wikipedia, using WordPiece tokens and token + segment + position embeddings.

Next session

Fine-tuning hands-on: PEFT, LoRA, QLoRA and quantisation. Re-read the paper's Figure 1 and Section 3 before then.

SESSION 12

Sunday, 12 July 2026

Fine-Tuning: PEFT, LoRA & QLoRA

Fine-Tuning Notes – Session 12 (12 July 2026)

Oct 14, 2026 · @VIjay

1. What fine-tuning is

Fine-tuning = extra training of a pre-trained model on your own domain examples so it gains specialised skills, style or behaviour. You start from an existing model; you never build one from scratch.

Analogies from class:

- A graduate who does honours in one subject: broad knowledge, plus one specialism.
- A doctor who already knows medicine and is trained further in heart surgery.

Banking-bot example: before fine-tuning, "What is a savings account?" gets a generic answer. After fine-tuning on the bank's material, it uses the bank's products, policies, terminology and tone.

When to fine-tune (vs. RAG)

1. **Start with RAG** for organisation data: it's cheaper and easy to update.
2. If answers are still poor, **improve RAG first**: chunking, embeddings, hybrid search, re-ranking.
3. **Fine-tune only when** you need behaviour RAG can't give: a specific style or tone, output format, domain reasoning, consistent persona.

Full vs. partial fine-tuning

	Full fine-tuning	Partial (PEFT)
What's trained	All weights (billions)	A tiny fraction of extra weights (typically under ~1–2%)
GPU/memory	Very high	Low
Cost/time	Very high: billions of weights × many epochs	Much lower
Storage	A full model copy per task	Small adapter files
Forgetting old knowledge	Higher risk	Lower risk (base weights frozen)
Use in practice	Rare	Default choice

2. PEFT, LoRA and QLoRA

PEFT (Parameter-Efficient Fine-Tuning) is the umbrella term for methods that freeze the pre-trained model and train only a small number of new parameters. LoRA and QLoRA are the most used; others include adapters, prefix tuning and prompt tuning.

LoRA (Low-Rank Adaptation)

Keep the original weight matrix W **frozen** and learn a small low-rank update made of two thin matrices, A and B:

```
W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d\times r},\; A \in \mathbb{R}^{r\times k},\; r \ll d,k
```

Worked example: a 1,000 × 1,000 layer has 1,000,000 weights. With rank r = 10, LoRA trains 1,000 × 10 + 10 × 1,000 = **20,000** weights (2% of the layer). Only these are updated by backprop each epoch.

Analogy: instead of rewriting a 1,000-page book, add a 10-page supplement.

Key hyperparameters: `r` (rank, e.g. 8–64), `lora_alpha` (scaling, e.g. 16), `lora_dropout` (e.g. 0.05–0.1), `target_modules` (which layers get adapters, usually attention projections such as q_proj and v_proj).

QLoRA (Quantized LoRA)

1. **Quantise** the frozen base model to **4-bit** (NF4), shrinking its memory.

2. Add **LoRA adapters** (trained in 16-bit) on top.
3. Train only the adapters.

Memory for a 7B-parameter model (weights only):

Precision	Size
32-bit float	~28 GB
16-bit	~14 GB
4-bit (QLoRA base)	~3.5 GB

This is how large models can be fine-tuned on a single GPU.

LoRA vs. QLoRA

	LoRA	QLoRA
GPU memory	Low	Very low
Training speed	Faster	Somewhat slower (weights are de-quantised on the fly)
Quality	Best	Very close to LoRA
Complexity	Medium	Higher
Choose when	You have enough GPU memory and want maximum accuracy	Memory is the bottleneck or the model is very large

3. Training methods

These are *what* you train on; LoRA and QLoRA are *how* cheaply you train. Any of them can be combined with LoRA/QLoRA.

Method	Training data	What it teaches	Use when
SFT – supervised fine-tuning	(prompt, ideal answer) pairs	Imitate the correct answers: format, tone, domain knowledge	Q&A bots, structured output, SQL/code generation, legal/medical assistants

Method	Training data	What it teaches	Use when
Full fine-tuning	Same data, but all weights updated	Deep behaviour change	Rare: needs lots of data and compute
DPO – Direct Preference Optimization	(prompt, chosen answer, rejected answer)	Prefer better answers over worse ones, without a separate reward model	You have comparison data and want alignment
ORPO – Odds Ratio Preference Optimization	Same preference triples	SFT + preference learning in one stage, with no reference model	A simpler, cheaper alternative to SFT → DPO
RLHF – reinforcement learning from human feedback	Human rankings → a reward model → RL (e.g. PPO)	Maximise human-judged quality via rewards and penalties	Large-scale alignment (how ChatGPT was tuned)
RFT – reinforcement fine-tuning	Prompts + a grader/reward function	Improve reasoning on tasks with checkable answers	Domain reasoning with automatic scoring

Preference example (DPO/ORPO):

- Rejected: "Your claim is rejected."
- Chosen: "Unfortunately your claim can't be approved because a required document is missing."

The model learns to prefer the clearer, more helpful style.

RLHF analogy: a game where completing a level earns a reward and failing costs a penalty, or training a dog with treats. Humans compare answers (A is better than B); a reward model learns those preferences; RL pushes the model toward higher reward.

DPO loss (for reference; "w" = chosen, "l" = rejected):

```
\mathcal{L}_{DPO} = -\log \sigma\Big(\beta\big[\log\tfrac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log\tfrac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\big]\Big)
```

4. Hands-on: tools and code

Choosing a pre-trained model on Hugging Face

1. Go to **Models** (~2.9M models) → filter by **Task** (e.g. question answering, text generation, summarisation, translation, image classification).
2. **Sort by "Most downloaded"** and shortlist the top few (e.g. DistilBERT for extractive QA, Llama 3.1 8B Instruct for generation).
3. Check the **model card**, licence, size (e.g. Llama 3.1 8B ≈ 16 GB of weights) and whether your GPU can hold it.
4. Create an **access token** (Settings → Access Tokens) for gated models like Llama.

No-code: Hugging Face AutoTrain

Spaces → duplicate **AutoTrain Advanced** → choose a task (LLM SFT, ORPO, DPO, reward modelling…) → base model → upload your dataset → set parameters (PEFT/LoRA on or off = LoRA vs. QLoRA, epochs, learning rate, batch size) → **Start training**. Requires paid GPU hardware (billed per hour).

Code: QLoRA + SFT with `transformers`, `peft`, `trl`

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

model_id = "meta-llama/Llama-3.1-8B-Instruct"
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_compute_dtype=torch.bfloat16)        #
QLoRA: 4-bit base
model = AutoModelForCausalLM.from_pretrained(model_id,
quantization_config=bnb, device_map="auto")
tokenizer = AutoTokenizer.from_pretrained(model_id)

lora = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.1,
                  target_modules=["q_proj", "v_proj"], task_type="CAUSAL_LM")

dataset = load_dataset("json", data_files="train.jsonl")["train"]  #
{"messages": [...]} chat format

trainer = SFTTrainer(
    model=model, train_dataset=dataset, peft_config=lora,
```

```
    args=SFTConfig(output_dir="out", num_train_epochs=3, learning_rate=2e-4,
                   per_device_train_batch_size=4, logging_steps=10),
)
trainer.train()
trainer.save_model("out/adapter")      # saves only the small LoRA adapter
```

The lecturer's 8B-parameter fine-tune took ~15 hours and ~32 GB of RAM on Colab Pro. The free Colab GPU crashed. For real runs, use rented GPUs (RunPod H100s, Hugging Face hardware, cloud VMs).

Managed options

- **Azure AI Foundry:** pick a model from the catalogue → Fine-tune → choose SFT/DPO/RFT → upload data. Microsoft's docs list typical uses: domain specialisation, task performance, style/tone, instruction following, compliance/safety, language/culture.
- **OpenAI platform:** the self-serve fine-tuning UI shown in class was no longer available to the lecturer's account. Check OpenAI's current fine-tuning docs before relying on it.

5. Prompting vs. RAG vs. fine-tuning vs. agentic AI

	Prompt engineering	RAG	Fine-tuning	Agentic AI
Best for	Instructions, persona, format	Knowledge from private/organisation documents	Behaviour, style, domain skills	Autonomous multi-step work, tool use
Uses organisation data	Very limited (only what fits in the prompt)	Yes	Yes (baked into weights)	Yes (via RAG/tools)
Latest/changing information	No	Yes: re-index new or edited docs	No: needs retraining	Yes: via tools, MCP, RAG
Add/update/delete data	No	Yes	No	Yes
Swap the LLM	Easy	Easy (model-agnostic)	Hard: redo fine-tuning per model	Easy (model is one component)

	Prompt engineering	RAG	Fine-tuning	Agentic AI
Cost	Very low	Low (embedding + retrieval)	Medium–high	Variable
Complex automation	No	No	No	Yes

Order of attack: prompt → RAG → better RAG (chunking, hybrid search, re-ranking) → fine-tuning → agents where workflows need tools and decisions.

6. Q&A highlights

Question	Answer
Is fine-tuning the same as hyperparameter tuning?	No. Hyperparameter tuning searches settings (learning rate, depth…). Fine-tuning continues training the model's weights on new data.
Is LoRA the same as transfer learning?	It's a form of transfer learning: reuse a pre-trained model and adapt it, here by training small adapters instead of the whole model.
How is the rank r chosen?	It's a hyperparameter (commonly 4–64), tuned on validation results, not random.
Does LoRA add a new dense layer on top?	No. It adds a parallel low-rank update (B·A) to existing weight matrices, usually the attention projections.
In QLoRA, is the parameter count the same?	Yes. The base model keeps its parameters; each is stored in 4 bits instead of 16/32, which shrinks memory.
Fine-tune an open model to cut API costs?	Fine-tuning itself costs GPU time. It *can* pay off at high volume by replacing a large API model with a smaller tuned one, but it isn't automatically cheaper. Start with RAG and prompt optimisation.
Can the free Colab T4 fine-tune an 8B model?	Rarely; QLoRA on very small models might fit. Use Colab Pro, RunPod or cloud GPUs.

Question	Answer
How does RAG handle new documents?	Re-index on a schedule (e.g. Azure AI Search indexer schedules), covered in the cloud and project sessions.
Will RAG evaluation be covered?	Yes, next week: context precision, recall, relevancy, faithfulness and more.
Will MCP be covered?	Yes, it's mandatory for interviews and is coming soon.

7. Expert corrections to the lecture

1. **SFT doesn't "pick the answer that appears most often".** It trains the model on curated (prompt, ideal response) pairs so it learns to produce those responses.
2. **DPO isn't "SFT choosing the preferred answer".** It optimises directly on chosen-vs-rejected pairs against a frozen reference model, with no reward model.
3. **ORPO doesn't "give reasons for rejecting",** and it isn't "the opposite of DPO". It merges SFT and preference learning into one loss using an odds-ratio penalty, without a reference model.
4. **RLHF doesn't involve humans writing reasons.** Humans rank or compare outputs, a reward model learns from those rankings, and RL (e.g. PPO) optimises the policy.
5. **"GT" isn't a standard term.** Full fine-tuning updates all weights on (usually supervised) data. "Unsupervised" further training is *continued pre-training*.
6. **LoRA doesn't add new neurons on top of the frozen model.** It adds low-rank matrices *alongside* existing weight matrices, and they can even be merged back after training.
7. **LoRA arithmetic:** 20,000 / 1,000,000 = **2%** (the lecture also said 0.02%).
8. **Quantisation isn't rounding decimals** (10.812… → 10.81). It maps weights to a low-bit format such as 4-bit NF4. Memory for 7B weights: ~28 GB (fp32) → ~3.5 GB (4-bit).
9. **QLoRA is slower per training step** because 4-bit weights are de-quantised during compute, not just because of a one-time conversion. Its quality is usually very close to 16-bit LoRA.
10. **Swapping models is easy in RAG.** The lecture's table said you can't change the model in RAG but can in fine-tuning; in reality RAG is model-agnostic, while fine-tuning must be redone per base model.
11. **PEFT includes more than LoRA and QLoRA:** adapters, prefix tuning, prompt tuning and IA^3 too.
12. **"Fine-tuning to save cost is 100% wrong" is too absolute.** Distilling into a smaller fine-tuned model can cut inference costs at scale; it depends on volume and quality needs.

8. Interview quick revision

> **What is fine-tuning and when would you use it?** Further training a pre-trained model on domain examples to change its behaviour, style or task skill. I'd first use prompting and RAG (cheaper, easy to update with new data), and fine-tune only when RAG can't deliver the required format, tone or domain reasoning.

> **What is PEFT / LoRA / QLoRA?** PEFT freezes the base model and trains a small set of new parameters. LoRA learns a low-rank update W + BA (often around 1–2% of weights or less). QLoRA does the same on a 4-bit-quantised base model, so large models fit on a single GPU, with near-LoRA quality but somewhat slower training.

> **SFT vs. DPO vs. ORPO vs. RLHF?** SFT learns from ideal answers. DPO learns from chosen-vs-rejected pairs without a reward model. ORPO combines SFT and preference learning in one step. RLHF trains a reward model from human rankings and optimises with RL.

Practice

- [ ] Browse Hugging Face → Models → Task filter → sort by downloads; shortlist a model for your domain
- [ ] Run the QLoRA + SFT script on a small model (e.g. a 1–3B instruct model) with 100 Q&A pairs
- [ ] Write the prompt/RAG/fine-tuning/agent comparison table from memory

Next: evaluation metrics, tool and function calling, memory, MCP and A2A.

SESSION 13

Sunday, 19 July 2026

RAG Evaluation Metrics

RAG Evaluation Notes – Session 13 (19 July 2026)

Oct 14, 2026 · @VIjay

1. Why and where to evaluate RAG

A RAG system succeeds only if it gives correct answers, from the right documents, without hallucination, consistently. Evaluation metrics are how you prove that to a client before going to production. Interviewers ask about them as often as RAG, agents and fine-tuning.

The two evaluation points

1. Indexing: raw data → load → chunk → embed → vector DB.
2. User query → **retriever** → relevant chunks (context). **Eval 1: retrieval quality.** Did we fetch the right documents?
3. Augmentation (prompt) → **LLM** → answer. **Eval 2: generation quality.** Is the answer correct and grounded?
4. Together: **end-to-end quality.** Did the user get the right answer?

School analogy: question paper = user query; textbook = vector DB; student = LLM; teacher = evaluator; marks = eval score.

The seven metrics to remember

Stage	Metric	Question it answers
Retrieval	Contextual precision	Are the relevant chunks ranked at the top?
Retrieval	Contextual recall	Did retrieval bring back *everything* needed for the ideal answer?
Retrieval	Contextual relevancy	How much of the retrieved text is relevant to the question?
Generation	Answer relevancy	Does the answer actually address the question?
Generation	Faithfulness	Is every claim in the answer supported by the retrieved context?
Generation	Hallucination	Does the answer contradict or invent beyond the given context?

Stage	Metric	Question it answers
Generation	LLM-as-a-judge (G-Eval)	Does it meet *your* custom criteria (business rules, tone, safety)?

2. Retrieval metrics

Contextual precision: "Are the retrieved chunks useful, and ranked first?"

Example question: "What is Azure OpenAI?"

Chunk	Relevant?
Azure OpenAI uses Azure infrastructure (Cosmos DB, storage…)	Yes
Azure virtual machine pricing	No
Azure OpenAI provides GPT models	Yes

Simple form taught in class:

```
\text{Precision} = \frac{\text{relevant retrieved chunks}}{\text{total retrieved chunks}} = \frac{2}{3} \approx 0.67
```

DeepEval's version is rank-weighted: it averages precision@k at each relevant position, so relevant chunks ranked lower are penalised. In the class demo, relevant chunks at ranks 2 and 3 of 5 gave (1/2 + 2/3) / 2 ≈ **0.58**.

Interpretation bands (rule of thumb): 0.9–1.0 excellent · 0.7–0.9 good · below 0.7 too much noise, fix retrieval.

Contextual recall: "Did we retrieve all the important information?"

The client's **golden answer** to "What is Azure OpenAI?" covers three points: (1) Azure's infrastructure, (2) GPT models, (3) enterprise security. Retrieval found 1 and 2 but missed 3.

```
\text{Recall} = \frac{\text{golden-answer facts supported by retrieved context}}{\text{total golden-answer facts}} = \frac{2}{3} \approx 0.67
```

Low recall means retrieval missed important data, so the LLM can't use it. Check chunking (size, overlap) and embeddings. **High recall** means the retrieved content covers everything required.

Precision vs. recall example: 10 documents retrieved, 8 relevant → precision 8/10 = 80%. But 20 relevant documents exist in the database → recall 8/20 = 40%.

Contextual relevancy: "How relevant is the retrieved text to the question?"

Question: "What is Copilot Studio?" Retrieved: "Copilot Studio enables low-code AI agents" (relevant) and "Microsoft campus location details" (irrelevant).

```
\text{Contextual relevancy} = \frac{\text{relevant statements in retrieved context}}{\text{total statements}} = \frac{4}{5} = 0.8
```

Higher means retrieval brings focused content instead of noise.

Which matters more depends on the domain

Domain	Costlier mistake	Prioritise
Healthcare, safety	Missing critical information (false negatives)	**Recall**
Customer chatbots, finance/insurance answers	Showing irrelevant or wrong information (false positives)	**Precision**

Reminder from ML: precision = TP / (TP + FP); recall = TP / (TP + FN).

3. Generation metrics

Faithfulness: "Is the answer supported by the retrieved documents?"

Context: "Azure OpenAI supports GPT-4."

- Answer A: "Azure OpenAI supports GPT-4" → **faithful**
- Answer B: "Azure OpenAI supports GPT-5" → **not faithful** (the claim isn't in the context)

```
\text{Faithfulness} = \frac{\text{claims supported by the retrieved context}}{\text{total claims in the answer}}
```

The evaluator (1) breaks the answer into claims, (2) checks each claim against the context, (3) computes the ratio. Closer to 1 is better. In the demo, both claims in the answer to "What is AI?" were supported, scoring 1.0.

Answer relevancy: "Did the answer actually address the question?"

- "What is the capital of India?" → "New Delhi is the capital of India" → highly relevant
- "What is the capital of India?" → "India is a beautiful country" → irrelevant

Two common implementations:

- **DeepEval:** an LLM splits the answer into statements and scores relevant statements / total statements.
- **RAGAS:** an LLM generates questions from the answer, then measures **cosine similarity** between those and the original question (cos 0° = 1 means identical meaning).

Hallucination: "Did the LLM invent or contradict information?"

Context: "Microsoft's CEO is Satya Nadella." Answer: "Microsoft's CEO is Elon Musk" → hallucination.

```
\text{Hallucination rate} = \frac{\text{unsupported or contradicting claims}}
{\text{total claims}} \quad\text{e.g. } \frac{2}{10} = 20\%
```

Lower is better. Production RAG typically targets under ~5% hallucination; 20% isn't acceptable for enterprise use. In DeepEval, `HallucinationMetric` scores the fraction of provided contexts the output contradicts (0 = best), and the threshold is the *maximum* allowed.

LLM-as-a-judge (G-Eval): custom criteria

Humans can't review 100,000 answers a day, so a strong LLM grades answers against **your** rules, such as faithfulness, relevance, correctness, safety and business logic, and explains its reasoning.

Hospital example of business rules a judge can check: identify the patient's problem → route to the right department (ENT, physician…) → check each doctor's calendar (max 10 patients/day) → honour a preferred doctor if free, otherwise offer an alternative or the next day.

Best practices:

- Use a strong judge model with temperature 0.
- Ask for structured (JSON) output with a reason.
- Calibrate against human labels and audit periodically.
- Watch for judge biases: position bias (preferring the first answer), verbosity bias (preferring longer answers), self-preference.

4. DeepEval code

Setup from the class notebook: a 10-row CSV (AI, ML, DL, NLP, photosynthesis, quantum mechanics…) → LangChain `Document`s with ID/title metadata → Chroma (`text-embedding-3-small`, cosine HNSW) → retriever with `similarity_score_threshold` (0.3, k = 5) → RAG chain with `gpt-4o-mini`. "What is AI?" retrieved 3 chunks; "How do plants survive?" retrieved 1 (photosynthesis).

```
!pip install deepeval ragas

from deepeval import evaluate
from deepeval.test_case import LLMTestCase, LLMTestCaseParams
from deepeval.metrics import (
    ContextualPrecisionMetric, ContextualRecallMetric,
ContextualRelevancyMetric,
    AnswerRelevancyMetric, FaithfulnessMetric, HallucinationMetric, GEval,
)

question = "What is AI?"
result = rag_chain_with_sources.invoke(question)
retrieved = [d.page_content for d in result["context"]]

test_case = LLMTestCase(
    input=question,
    actual_output=result["answer"],
    expected_output=("AI, also known as artificial intelligence, is used to
build complex systems "
                     "for applications like virtual assistants, robotics and
autonomous vehicles."),  # golden answer
    retrieval_context=retrieved,   # used by contextual / faithfulness /
answer metrics
    context=retrieved,             # used by HallucinationMetric
)

metrics = [
    ContextualPrecisionMetric(threshold=0.5, model="gpt-4o",
include_reason=True),
    ContextualRecallMetric(threshold=0.5, model="gpt-4o",
include_reason=True),
    ContextualRelevancyMetric(threshold=0.5, model="gpt-4o",
include_reason=True),
    AnswerRelevancyMetric(threshold=0.5, model="gpt-4o",
include_reason=True),
    FaithfulnessMetric(threshold=0.5, model="gpt-4o", include_reason=True),
    HallucinationMetric(threshold=0.05, model="gpt-4o", include_reason=True),
# max 5% allowed
```

```
    GEval(
        name="RAG Fact Checker",
        evaluation_steps=[
            "Create a list of statements from the actual output.",
            "Check each statement is relevant to the input; penalise
irrelevant ones.",
            "Compare with the expected output; penalise missing or factually
wrong statements.",
            "Check statements are grounded in the retrieval context; penalise
ungrounded ones.",
            "Penalise anything invented or nonsensical given the input and
context.",
        ],
        evaluation_params=[LLMTestCaseParams.INPUT,
LLMTestCaseParams.ACTUAL_OUTPUT,
                           LLMTestCaseParams.EXPECTED_OUTPUT,
LLMTestCaseParams.RETRIEVAL_CONTEXT],
        model="gpt-4o", threshold=0.5,
    ),   # give either `criteria` or `evaluation_steps`, not both
]

evaluate(test_cases=[test_case], metrics=metrics)
```

Class results:

Metric	Score	Outcome
Contextual precision	0.58	Passed at 0.5; **failed** when the threshold was raised to 0.9. Irrelevant ML/NLP chunks outranked a relevant one
Faithfulness	1.0	Both claims supported
Answer relevancy	1.0	Fully on-topic
G-Eval fact checker	0.88	Passed

Tip: collect each metric's score, success flag and reason into a pandas DataFrame for one summary table per test set.

5. DeepEval vs. RAGAS, golden datasets, fixing low scores

	DeepEval	RAGAS (Retrieval-Augmented Generation Assessment)
Type	Open-source LLM testing framework ("pytest for LLMs")	Open-source evaluation library, RAG-first
Scope	RAG, chatbots, agents, any LLM app	RAG primarily; has added agent/tool metrics
Metrics	All seven above, plus task completion, custom G-Eval, safety	Context precision/recall, faithfulness, answer relevancy…
CI/CD	Strong (pytest integration)	Basic
Lecturer's pick	**DeepEval** for enterprise and agentic work	Fine for RAG-only evaluation

Golden dataset

Ask the client for a **golden dataset**: representative questions plus the answers they expect, written by domain experts. You can't know the right answers in an unfamiliar domain (e.g. healthcare) yourself. It powers recall, the expected-output comparison and G-Eval.

When scores are low

1. **Retrieval metrics low:** revisit chunking (size, overlap, strategy), the embedding model, and the retrieval method (hybrid search, re-ranking, k and threshold).
2. **Faithfulness or hallucination poor:** tighten the augmentation prompt ("answer only from context; otherwise say you don't know"), add few-shot examples or chain-of-thought, add guardrails.
3. **Still poor:** try a stronger LLM; as a last resort, fine-tune (LoRA/QLoRA).

Latency is **not** measured by these metrics. Track it with observability tools (LangSmith, App Insights): time to first token, tokens per second, end-to-end time.

6. Q&A highlights

Question	Answer
Isn't contextual relevancy the same as precision?	They're close. Precision (DeepEval) is rank-weighted relevance of retrieved *nodes* judged against the expected answer. Relevancy is the share of retrieved *statements* relevant to the question.
Should I optimise for recall or precision?	Retrieve broadly first (recall), then rank and filter for precision. The final emphasis depends on the domain (see Section 2).
How is recall computed if the knowledge base is huge?	Against the golden answer: what fraction of its facts appear in the retrieved context. You never need to count every relevant chunk in the database.
Who decides which chunks are relevant?	An evaluator LLM, guided by the metric's prompt template, inside DeepEval/RAGAS.
Faithfulness vs. hallucination?	Faithfulness = share of claims supported (higher is better). Hallucination = share contradicted or invented (lower is better). They're related but not identical.
How do I evaluate an agent?	The same way: an agent has model, tools (data, MCP), memory and planning. Log its inputs, outputs and contexts, then apply DeepEval metrics (plus task-completion and tool-use metrics).
Does DeepEval measure latency?	No. Use observability tools.
Upcoming topics	Tool and function calling, guardrails, MCP, A2A, optimisation. Course completion targeted for mid-August.

7. Expert corrections to the lecture

1. **DeepEval's contextual precision is rank-weighted, not simply relevant ÷ total.** That's why 2 relevant of 5 scored 0.58 instead of 0.40 or 0.67.

2. **DeepEval's contextual recall** measures how much of the *expected output* is attributable to the retrieval context. It isn't relevant chunks ÷ all relevant chunks in the database.
3. **Contextual relevancy isn't "the same as precision".** Relevancy judges each retrieved statement against the *input*. Precision judges ranking of relevant nodes against the *expected output*.
4. **Answer relevancy via cosine similarity is RAGAS's method.** DeepEval uses an LLM to classify statements as relevant or not.
5. **DeepEval's hallucination score: lower is better** (0 = no contradictions); the threshold is a maximum. A score of 1.0 would mean every context was contradicted. The lecture's "100% means 0% hallucination" reading was backwards.
6. **Faithfulness and hallucination aren't exact reciprocals.** One counts supported claims; the other counts contradicted contexts.
7. **The confusion-matrix sketch mixed up cells.** Precision = TP/(TP+FP); recall = TP/(TP+FN).
8. **"Finance → precision" isn't universal.** Fraud detection usually prioritises recall. Choose by the cost of each error type.
9. **A golden dataset isn't RLHF.** It's ground-truth Q&A used for evaluation. RLHF is a training method.
10. **Recall example:** 8 of 20 = 40%. The lecture's "20% missing" wording was garbled.
11. **RAGAS now has agent and tool-use metrics.** "RAGAS can't evaluate agents" is outdated, though DeepEval remains broader.
12. **The thresholds are rules of thumb, not standards** (≥0.9 excellent, <5% hallucination). Agree targets with the client.

8. Interview quick revision

> **How do you evaluate a RAG system?** At two points. Retrieval: contextual precision (relevant chunks ranked first), contextual recall (does the context cover the golden answer), contextual relevancy (how much retrieved text is on-topic). Generation: answer relevancy, faithfulness (claims grounded in context), hallucination (contradictions; target under ~5%), plus LLM-as-a-judge (G-Eval) for custom business rules. I run these with DeepEval against a client-provided golden dataset, set thresholds, and wire them into CI/CD.

> **Your scores are low. What do you do?** Diagnose by stage. Poor retrieval metrics → fix chunking, embeddings, hybrid search, re-ranking. Poor faithfulness or hallucination → tighten the prompt, add guardrails, try a stronger model. Fine-tune only as a last resort.

> **DeepEval or RAGAS?** RAGAS is RAG-focused and lightweight. DeepEval covers RAG, agents and custom judges with CI/CD integration, so I prefer it for enterprise systems.

Practice

- [ ] Run the class notebook; raise thresholds to 0.8–0.9 and see what fails
- [ ] Write a 10-question golden dataset for your own RAG project and evaluate it
- [ ] Add a G-Eval judge with one business rule specific to your domain

SESSION 14

Saturday, 25 July 2026

Low-Code RAG on Azure

Azure RAG Notes – Session 14 (25 July 2026)

Oct 14, 2026 · @VIjay

1. Azure GenAI services and the reference architecture

This session built a working RAG chatbot on Azure with no code: Blob Storage (raw files) → AI Search (chunk, embed, index) → Foundry (models, knowledge base, agent) → a published chat app. Pro-code versions come in the project batch. Knowing one cloud well is enough for most GenAI interviews.

Reference flow: user → front end (web app, Teams, Copilot Studio) → application layer (Container Apps / App Service / API) → Azure OpenAI models in Foundry ↔ Azure AI Search (index) ← Azure Blob Storage (documents). Cosmos DB stores chat history; Key Vault holds secrets; App Insights / Log Analytics monitor; API Management governs access.

Service	Role in a GenAI app
Azure OpenAI / Foundry models	The "brain": LLMs (GPT-4.1, GPT-5 family, o-series, plus partner models like Llama, DeepSeek, Grok, Claude) and embedding models
Azure AI Foundry	Development platform: deploy models, build agents and RAG, connect knowledge, evaluate, fine-tune, add guardrails
Azure AI Search	Search and vector index: chunking, embedding, keyword, vector, hybrid and semantic retrieval
Azure Storage (Blob / Data Lake)	Stores raw files: PDF, Excel, images, policies
Cosmos DB	Conversation history / memory, app data
Key Vault	Secrets: API keys, passwords, certificates
API Management (APIM)	Gateway: auth, rate limits, logging, cost control for model/API calls
Application Insights / Log Analytics	Usage, failures, latency, logs
Container Apps / App Service	Host the API or web front end

2. Step-by-step setup

0. Account

1. Go to the Azure portal → **Try Azure for free** (not pay-as-you-go).
2. Sign in with a personal email; verify your phone; add address and card details. A small (~₹2) verification charge is refunded.
3. You get **$200 credit valid for 30 days**.

1. Create Azure AI Search

1. Search **AI Search** → Create.
2. **Resource group:** create new, lowercase, no underscores (e.g. `sahana`). Every later resource goes in this group.
3. **Service name:** lowercase, no spaces (e.g. `sahanaai`). **Region: Sweden Central**, chosen for model availability. Use the same region everywhere.
4. **Pricing tier: change to Basic** (~~$75/month, 15 GB). The default Standard ($250/month)~~ would exceed the $200 credit.
5. Networking: **Public** for this demo (use **Private** endpoints at work). Keep other defaults → Review + create.

2. Create a Storage account

1. Search **Storage accounts** → Create → same resource group, unique lowercase name (e.g. `sahanastorage`), Sweden Central.
2. Primary service: **Azure Blob Storage or Data Lake Storage Gen2**. Performance: Standard. Other settings default.
3. **Settings → Configuration → "Allow Blob anonymous access" → Enabled → Save.** Check it again; the first save sometimes doesn't stick.
4. **Data storage → Containers → + Container** (e.g. `dummydata`); under Advanced set anonymous access level to **Container**.
5. Open the container → **Upload** the three PDFs from "Assignment for learning / Assignment 2".
6. Verify: click a blob → copy its URL → open in a browser. If the PDF opens, the upload worked.

3. Create the Foundry resource (Azure OpenAI)

1. Search **Azure OpenAI** → Create → choose **Foundry** (recommended).

2. Same resource group; name `sahanaopenai`; region Sweden Central; project name `rag-project` (hyphens, no underscores).
3. Keep defaults (public network, no outbound restrictions) → Create → **Go to Foundry portal**.

4. Deploy two models in Foundry

1. **Deployments → Deploy model → Deploy base model → All models**.
2. Deploy **`text-embedding-3-small`** (embeddings, used by AI Search) with Global Standard / default settings.
3. Deploy a chat model, e.g. **`gpt-5-mini`**; some older models such as GPT-4o-mini showed as deprecated.
4. Test the chat model in the **Playground**, then copy the sample Python code (endpoint + API key) for later pro-code work.

3. Indexing with AI Search

AI Search → your service → Import data. Sources include Blob Storage, Data Lake, Cosmos DB, Azure SQL, Table storage, OneLake, SharePoint, OneDrive, Azure Files, plus Logic Apps connectors (e.g. Amazon S3, Dropbox, SFTP).

Three wizard modes:

Mode	What it builds	Use for
Keyword search	Text index; optional AI skills (key phrases, entities, OCR)	Product/site search, recommendations ("mobile phone" → phone results)
RAG	Chunked text + **vector embeddings** (via your Foundry embedding deployment)	Text-only RAG chatbots
Multimodal RAG	Also extracts and vectorises images	Documents with figures and scans

RAG wizard steps:

1. Source: subscription → storage account → container (`dummydata`). Parsing mode: Default.
2. Vectorise text: Foundry project `rag-project` → `text-embedding-3-small` → API key → acknowledge pricing.
3. Image vectorisation: skip (that's the multimodal option).

4. Schedule: **Once** for the demo. For live data choose hourly/daily/custom; the indexer then re-chunks and re-embeds new files automatically, at extra cost.
5. Create, then wait for the indexer.

Result: 3 PDFs → **244 chunks** (Search management → Indexers shows 3 documents succeeded; Indexes shows the document count and vector index size).

Test in Search explorer: queries like "self-attention" or "what is few-shot learning" return the top matching chunks with scores, the chunk text and its stored `text_vector` field. **View → JSON view** shows the query body, which pro-code apps reuse.

4. Foundry: knowledge base, agent, publishing

Create a knowledge base (Foundry IQ)

1. Foundry → **Knowledge** → connect to your AI Search service (API key).
2. **Create knowledge base:** chat model `gpt-5-mini`, output mode answer synthesis, minimal retrieval effort. Retrieval instructions are where guardrail-style rules can go (e.g. "never reveal staff personal details").
3. **Add source → Azure AI Search index** → pick the **RAG** index (not the keyword-search one) → save.

Blob Storage, SharePoint, Fabric and MCP servers can also be sources. Use MCP for stores without a native connector (e.g. Postgres/pgvector). For RAG, point at the *embedded* AI Search index rather than raw blobs.

Create an agent and connect the knowledge

1. **Build an agent** (or Playground → Agents → New agent) → name it.
2. **Instructions** (this is the augmentation prompt):

```
You are an AI assistant that helps people find information based on the given
knowledge base.
If the answer is not in the context, simply say "I don't know."
Always provide citations showing which document the response came from.
```

3. **Knowledge → Add → your knowledge base → Save.** If prompted, assign yourself the required Foundry role and refresh.
4. **Test:** "What is self-attention?" → grounded answer with numbered citations. An off-topic question gets "I don't know."

Other agent options: tools (web search, MCP, SharePoint, file search, code interpreter), memory (Cosmos DB-backed stores), guardrails, voice mode, evaluation.

Publishing

- **Agent → Publish:** preview web page, Microsoft Teams, Microsoft 365 Copilot. This is the fastest low-code route to users.
- **App Service:** Create → Web App → same resource group, Code, Python 3.12/3.13, Linux, Sweden Central. A new web app shows a "waiting for your content" page until you **deploy chat-app code** to it (e.g. Azure's "chat with your data" sample or Foundry's deploy-to-web-app option). That's the pro-code step covered in the project batch.

5. Cost and security practices

Cost

- AI Search bills by provisioned tier per hour **whether or not you use it**. Delete the service (or the whole resource group) when done.
- Embedding, chat tokens, indexer runs and App Service plans add usage costs. Scheduled re-indexing multiplies them.
- After the 30-day trial, resources stop unless you upgrade to pay-as-you-go. Then charges apply, so set a **budget alert** in Cost Management.
- Use the **Azure pricing calculator** to quote clients. A production RAG with APIM and private networking can run to thousands of dollars a month.
- **Cleanup:** delete the resource group to remove everything at once.

Security (what changes at work)

- Use **private endpoints / VNets** instead of public network access.
- **Don't enable anonymous Blob access.** Let AI Search read storage via **managed identity** (or a key) with RBAC.
- Keep keys in **Key Vault**; never share API keys with clients or commit them to code.
- Put **APIM** in front of model endpoints for auth, rate and token limits, and logging. Monitor with App Insights.
- Restrict who can use the app (Entra ID users and roles) and apply guardrails/content filters.

6. Q&A highlights

Question	Answer
Do I need to know AWS and GCP too?	One cloud known well is enough. Mention awareness of the others. AWS/GCP are covered in the project batch.

Question	Answer
Will the free trial auto-charge me?	Not during the trial. After 30 days you must upgrade to keep resources; then usage is billed.
Why Sweden Central?	It offers most Foundry models and AI services. Use one region for every resource.
Storage creation failed. Why?	Usually the name is taken, the primary service wasn't selected, or the region is missing. Fix it on the Basics tab.
Is the RAG index different from the keyword index?	Yes. The RAG index holds embeddings (`text_vector`); attach that one to the knowledge base.
How do I return 5 results instead of 3?	In the query JSON set `top` (and `k` for vector queries). `count: true` only returns the total match count.
Is Blob Storage a vector database?	No. It stores raw files; vectors live in the AI Search index. Agents can use blobs directly as files, but RAG needs the index.
Why does my agent say "not in the knowledge base"?	Check that the knowledge base uses the RAG index (not the keyword one), the chat model is set, and the agent's Knowledge section is connected and saved.
Can external people use my public app?	They can access the endpoint. Restrict it with authentication, private networking and token limits.
Do POCs get built in the portal at work?	Yes. Quick POCs are low-code; production uses pro-code pipelines over large datasets (e.g. 100k documents in SharePoint) with CI/CD.
What publish settings for a web app?	Publish: Code; Runtime: Python 3.12; OS: Linux; Region: Sweden Central.

7. Expert corrections to the lecture

1. **Anonymous Blob access isn't required for AI Search to read files.** The indexer can authenticate with a connection string or managed identity. Enabling anonymous/public access is a demo shortcut and a security risk.

2. **AI Search costs accrue while the service exists,** not only "when it's up and running" queries. Basic (~$75/month) bills hourly even when idle.
3. **Don't create extra accounts with other cards to collect more free credit.** It breaches Azure's free-account terms.
4. **Blob Storage isn't "like a vector database",** and AI Search and Storage aren't "inside" Azure OpenAI. They're separate services wired together.
5. **Parsing mode isn't chunking.** It controls how files are read (default, text, JSON, JSON lines, delimited). Chunking is done by the wizard's text-split skill.
6. **Search explorer shows stored chunk vectors** (`text_vector`), not your query converted into numbers.
7. **Semantic ranker scores range 0–4,** not percentages, and search scores aren't "% match". Compare them relatively.
8. **`count: true` doesn't set how many results come back.** Use `top` / `k`.
9. **An empty App Service web app isn't a chatbot.** You must deploy app code (or use the agent's Publish options); the "waiting for your content" page means nothing is deployed yet.
10. **Model availability changes by region and over time.** Check the Foundry model catalogue and retirement dates before choosing (GPT-4o-mini appeared deprecated in class).

8. Interview quick revision

> **Which Azure services have you used for GenAI?** Azure OpenAI models through Foundry (chat and embedding deployments, agents, evaluation, fine-tuning), Azure AI Search for chunking, vectorisation and hybrid/semantic retrieval, Blob Storage for source documents, Cosmos DB for chat history, Key Vault for secrets, APIM as the model gateway, and App Insights / Log Analytics for monitoring, hosted on App Service or Container Apps, with private endpoints in production.

> **Walk me through a RAG build on Azure.** Upload documents to Blob Storage. Use AI Search's import wizard (or an indexer pipeline) to chunk and embed them with text-embedding-3-small into a vector index, scheduled for refresh. Create a Foundry knowledge base on that index. Build an agent with grounding instructions and citations. Evaluate it, then publish to a web app or Teams behind authentication and APIM.

Practice

- [ ] Rebuild the whole pipeline from scratch using the recording
- [ ] Try the **multimodal RAG** wizard on a PDF with images
- [ ] Set a budget alert, then **delete the resource group** when finished

SESSION 15

Sunday, 26 July 2026

Graph RAG & Corrective RAG

Graph RAG & CRAG Notes – Session 15 (26 July 2026)

Oct 14, 2026 · @VIjay

1. Opening Q&A and low code vs. pro code

This session moved from low code (Session 14's portal build) to pro code, and introduced the last two RAG types: Graph RAG and Corrective RAG (CRAG). The live Graph RAG demo didn't complete (Azure model/endpoint errors), so the code is explained here and the run was postponed to the next session.

Role-based answers in one app (student question)

Scenario: one HR app serves employees, managers and directors, and each role may see only its own level of data (e.g. nobody may see VP or director performance ratings).

Lecturer's approach: add a **checkpoint** before answering:

1. Identify the requester and their designation.
2. If the question touches restricted data (e.g. a VP's rating), reply "Sorry, I'm not authorised to give this answer."
3. Offer a **ticket**: the request goes to an authoriser by email, who approves genuine requests (e.g. from a senior VP).
4. Multiple levels are possible; define the rules and parameters at design time.

See correction 1: in production this must be enforced in the data layer, not only in the prompt.

Low code vs. pro code

	Low code	Pro code
Tools	Azure portal wizards, Foundry agents, **Copilot Studio** (knowledge, skills, Dataverse, Power Automate)	Python calling Azure endpoints, API keys and models
Used for	POCs (proof of concept), quick demos to clients, simpler bots; Microsoft is pushing Copilot Studio heavily	Complex architectures and production systems

	Low code	Pro code
Who needs it	Everyone	GenAI developers and architects must know both

Session 14 follow-up: the App Service page still showed "waiting for your content" because no app code or data source was connected. That's covered in the project batch.

Status of the course: GenAI, agentic AI, cloud and fine-tuning are done. Remaining: MCP, A2A, tool and function calling, guardrails, then the capstone project.

2. Graph RAG

Graph RAG adds a knowledge graph to RAG, so retrieval can follow relationships between entities (multi-hop) instead of only finding similar text chunks. Use it when questions ask *how* things are connected, or need facts spread across many documents.

Why traditional RAG isn't enough

Vector search finds chunks that *look like* the question. It doesn't understand **deeper relationships** or connections that only appear when you join several facts together.

Example (multi-hop reasoning):

- Sundaram is a GenAI architect at Microsoft.
- Sundaram **reports to** Himanshu.
- Himanshu **reports to** Manoj.
- All three **work at** Microsoft.

"What is the relationship between Sundaram and Manoj?" No single chunk says it. Simple RAG might answer "both work at Microsoft". Graph RAG follows Sundaram → Himanshu → Manoj and answers: **Manoj is Sundaram's skip-level manager ("super boss")**.

Other examples from class:

- **Ramayana:** "How are Janaka and Dasharatha related?" Janaka's daughter Sita married Dasharatha's son Rama, so they're the bride's and groom's fathers (co-parents-in-law). The answer needs three hops.
- **Code bases:** graphs of which function calls which, used to understand legacy code.

Building blocks of a graph

Element	Meaning	Example
Node (entity)	A thing: person, product, place, organisation	Sundaram, Microsoft, Creek Hotel, Dubai
Relationship (edge)	A typed, directed link between two nodes	REPORTS_TO, WORKS_AT, IS_LOCATED_IN, HAS_FACILITY
Property	Key–value metadata on a node or edge	rating: 4, date: 2018-03-01

Hotel-review case study (Contoso Travels)

A travel agency has thousands of customer reviews for ~50–100 hotels and wants a recommendation chatbot.

Question	Simple RAG	Graph RAG
"What is the review of Creek Hotel in Dubai?"	Yes	Yes
"What facilities does Kensington Hotel provide?"	Yes	Yes
"Which hotels were reviewed by reviewer XYZ?"	Poor: matches spread across many reviews	Yes: follow REVIEWED edges
"Which hotels are near Creek Hotel?"	Poor	Yes, if location relationships are modelled
"List all hotels that include complimentary breakfast"	Poor: needs aggregation across all reviews	Yes: filter HAS_FACILITY edges

A short history

Year	Milestone
1998–2000	Google search with **PageRank** (a link-graph algorithm); 1 billion URLs indexed by June 2000
May 2012	Google **Knowledge Graph**: "Things, not strings" (Amit Singhal). Search "Taj Mahal" or "Marie Curie" and a facts panel appears

Year	Milestone
2017–2023	Transformers, then LLMs
2024	Microsoft Research's **GraphRAG** paper and open-source library popularise LLM-built knowledge graphs for RAG
2025–	Enterprise demand for Graph RAG grows

Knowledge graphs are an old idea; using LLMs to build and query them for RAG is new.

Benefits: relationship and multi-hop answers, more explainable retrieval paths, better answers to "why/how" and aggregate questions. **Costs:** LLM calls to extract entities, a graph DB to run, a schema to design, and more complexity than vector RAG.

3. Corrective RAG (CRAG)

CRAG is traditional RAG plus a self-check: after retrieval, it grades the retrieved documents and corrects the context before generating. It isn't a graph technique.

> Traditional RAG: retrieve → generate CRAG: retrieve → **validate** → **correct** → generate

Class example: stale data

Question: "How many employees work in the GenAI department?" An old chunk says **100**; the latest report says **150**. Plain RAG may answer 100. CRAG:

1. Checks the **source** of each retrieved chunk.
2. Scores **confidence**: how relevant and reliable is it?
3. Judges overall **retrieval quality**.
4. If it's weak or conflicting, **retrieves additional evidence** (newer documents, a re-query, or web search) before answering.

The result: "The department started with 100, but after recent hiring it has 150."

How the original CRAG paper does it (Yan et al., 2024)

A lightweight **retrieval evaluator** labels the retrieved set:

Verdict	Action
Correct	Refine the documents (split into strips, keep only relevant parts), then generate

Verdict	Action
Incorrect	Discard them and search elsewhere (originally web search)
Ambiguous	Combine refined documents with extra search results

In LangGraph this is usually built as nodes: retrieve → grade documents → (rewrite query + search) → generate.

Benefits and when to use it

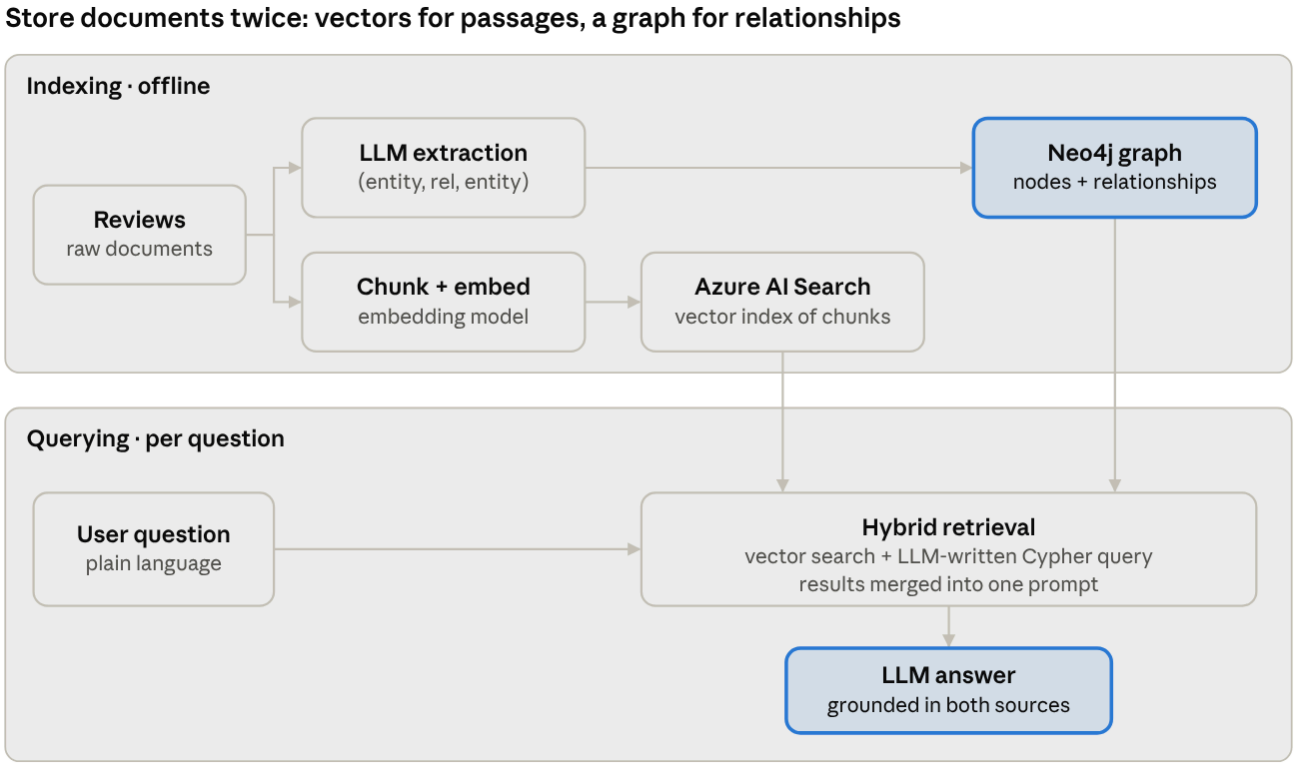
- **Less hallucination** and better accuracy from the built-in self-check.
- Suited to **low-tolerance domains**: banking, healthcare, legal, government, compliance and audit.
- **Trade-off:** extra LLM calls, so higher cost and latency.

Lecturer's one-liners: Graph RAG is the **detective** (finds relationships); CRAG is the **auditor** (verifies before answering).

"Why not just use evaluation metrics?" Evaluation (DeepEval, RAGAS) scores a system offline or in monitoring; it doesn't change the answer. CRAG acts **at query time**, fixing the context before the user sees a response. You still evaluate a CRAG system.

4. Graph RAG architecture on Azure + Neo4j

The class design is hybrid: the same data feeds Azure AI Search (chunks + embeddings) *and* Neo4j (entities + relationships). At question time the LLM draws on both before answering.



Hybrid Graph RAG on Azure · AI Search + Neo4j

1. **Indexing:** chunk and embed reviews into Azure AI Search, and in parallel have the LLM extract (entity, relationship, entity) triples and load them into Neo4j.
2. **Retrieval:** embed the question for vector search, and have the LLM translate it into a **Cypher** query run on Neo4j.
3. **Generation:** an aggregator combines passages and graph rows into one prompt; the LLM answers.

Neo4j and Cypher

Neo4j is the most widely used graph database: a property-graph store optimised for traversing relationships. It's available as a desktop app or **AuraDB** in the cloud (free tier: one instance).

Cypher is its SQL-like query language: patterns are drawn with ASCII arrows.

```
// SQL:    SELECT h.name FROM hotels h JOIN facilities f ... WHERE f.name =
'Pool'
// Cypher:
MATCH (h:Hotel)-[:HAS_FACILITY]->(f:Facility {name: 'Swimming pool'})
RETURN h.name

// Multi-hop: who is Sundaram's skip-level manager?
MATCH (:Person {name:'Sundaram'})-[:REPORTS_TO]->(:Person)-[:REPORTS_TO]->
(boss)
RETURN boss.name
```

LLMs write Cypher well when given the graph schema and a few examples (few-shot), so you rarely hand-write it.

5. Hands-on: Graph RAG code

Setup: VS Code + Jupyter (not Colab, in the lecturer's setup), five sample hotel reviews (Creek Hotel Dubai, Kingston Hotel Oslo…), Azure OpenAI in Foundry, and a free **Neo4j AuraDB** instance.

Step 1 – Neo4j AuraDB (free)

1. Sign up at Neo4j Aura → **Create a free instance** (one free instance at a time).
2. **Download the credentials .txt immediately.** The password is shown only once. If you miss it, delete the instance and create a new one.
3. The file contains `NEO4J_URI` (like `neo4j+s://xxxxxxxx.databases.neo4j.io`), `NEO4J_USERNAME` (`neo4j`) and `NEO4J_PASSWORD`.

Step 2 – `.env` (no quotes needed; never share or commit it)

```
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
AZURE_OPENAI_API_VERSION=2024-10-21
GPT_ENGINE=<your chat deployment name>
NEO4J_URI=neo4j+s://xxxxxxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=<password>
```

Step 3 – Connect to Azure OpenAI

```
%pip install openai python-dotenv neo4j      # in notebooks use %pip, not
!pip

import os
from openai import AzureOpenAI
from dotenv import load_dotenv
load_dotenv(override=True)    # override=True picks up edits to .env without
restarting

client = AzureOpenAI(
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
)
MODEL = os.getenv("GPT_ENGINE")   # must be the DEPLOYMENT name, not the
model family
```

Step 4 – Extract entity–relationship triples (few-shot prompt)

```
SYSTEM_NER = """You are a named-entity-recognition expert.
Identify named entities (person, place, thing) and the relationships between
them.
Entity types: Hotel, Location, Facility, CustomerType, Reviewer.
Relationships: IS_LOCATED_IN, HAS_FACILITY, HAS_CUSTOMER_TYPE, REVIEWED_BY.
Return ONLY tuples, one per line, in the form: (entity, RELATIONSHIP, entity)
Example:
(Creek Hotel, IS_LOCATED_IN, Dubai)
(Creek Hotel, HAS_FACILITY, Swimming pool)
(Creek Hotel, HAS_CUSTOMER_TYPE, Business)"""

def extract_triples(review_text: str) -> str:
    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "system", "content": SYSTEM_NER},
                  {"role": "user", "content":
f"Review:\n{review_text}\n\nReturn the tuples."}],
        temperature=0,     # omit for reasoning models that reject it (e.g.
gpt-5-mini)
    )
    return resp.choices[0].message.content
```

```
from pathlib import Path
ontology = [extract_triples(p.read_text(encoding="utf-8"))
            for p in sorted(Path("reviews").glob("*.txt"))]   # a loop scales
to thousands
```

Step 5 – Load the graph into Neo4j

The class asked the LLM to convert triples into Cypher. Writing parameterised `MERGE` statements in code is safer and avoids duplicate nodes:

```
import re
from neo4j import GraphDatabase

driver = GraphDatabase.driver(os.getenv("NEO4J_URI"),
                              auth=(os.getenv("NEO4J_USERNAME"),
os.getenv("NEO4J_PASSWORD")))

TRIPLE = re.compile(r"\(([^,]+),\s*([A-Z_]+),\s*([^)]+)\)")
ALLOWED = {"IS_LOCATED_IN", "HAS_FACILITY", "HAS_CUSTOMER_TYPE",
"REVIEWED_BY"}

with driver.session() as s:
    for block in ontology:
        for head, rel, tail in TRIPLE.findall(block):
            if rel in ALLOWED:   # relationship types can't be parameters, so
whitelist them
                s.run(f"MERGE (a:Entity {{name:$h}}) MERGE (b:Entity
{{name:$t}}) "
                      f"MERGE (a)-[:{rel}]->(b)", h=head.strip(),
t=tail.strip())
```

In class, the lecturer's sample graph had about 42 nodes plus their relationships (e.g. Kingston Hotel → HAS_FACILITY → …). Explore it in the Aura console's Query/Explore views.

Step 6 – Question → Cypher → answer

```
SCHEMA = "Nodes: (:Entity {name}). Relationships: IS_LOCATED_IN,
HAS_FACILITY, HAS_CUSTOMER_TYPE, REVIEWED_BY."

def to_cypher(question):
    prompt = (f"Schema: {SCHEMA}\nWrite ONE read-only Cypher query answering
the question. "
              f"Return only the query.\nQuestion: {question}")
    r = client.chat.completions.create(model=MODEL, messages=[{"role":
"user", "content": prompt}])
    return
r.choices[0].message.content.strip().strip("`").removeprefix("cypher").strip(
)

def graph_answer(question):
    cypher = to_cypher(question)
    with driver.session(default_access_mode="READ") as s:   # read-only:
blocks writes
        rows = [r.data() for r in s.run(cypher)]
    final = client.chat.completions.create(model=MODEL, messages=[
        {"role": "system", "content": "Answer only from the graph results. If
empty, say you don't know."},
        {"role": "user", "content": f"Question: {question}\nGraph results:
{rows}"}])
    return final.choices[0].message.content

graph_answer("Which hotels are visited by business customers?")
# e.g. MATCH (h)-[:HAS_CUSTOMER_TYPE]->(:Entity {name:'Business'}) RETURN
h.name
```

For richer answers, also pass vector-search chunks from Azure AI Search into the final prompt (hybrid Graph RAG). A student suggested this; the lecturer agreed.

Why the demo failed: "404 Resource not found"

The code wasn't the problem; the Azure connection was. Check these in order:

1. `azure_endpoint` is the resource base URL (`https://<name>.openai.azure.com/`), not a full `/chat/completions` URL.
2. `model=` is your **deployment name** exactly as shown in Foundry → Deployments.
3. The **API version** supports that model ("API version not supported" appeared for one student).
4. The notebook actually loaded the new `.env` values (`load_dotenv(override=True)` or restart the kernel). In class the old key stayed cached until reopening.

5. The model isn't **deprecated** and the subscription has **quota** for it. Free-trial quotas blocked several deployments (GPT-4o/4o-mini deprecated; GPT-5.1 "insufficient quota").

6. RAG types compared

Choose the RAG type **at design time** from the questions users will ask. Fine-tuning is a separate, later lever when *any* RAG type underperforms.

Type	Adds over basic RAG	Best for	Cost/complexity
Traditional (naive) RAG	—	Direct factual Q&A over documents	Low
Multimodal RAG	Images, tables, audio (summarised or embedded)	PDFs with charts, scans, mixed media	Medium
Agentic RAG	An agent decides when and where to retrieve; tools	Multi-source, multi-step tasks	Medium–high
Corrective RAG (CRAG)	Grades retrieval; re-retrieves or searches before answering	High-accuracy, low-tolerance domains	Medium (extra LLM calls)
Graph RAG	Knowledge graph of entities and relationships; graph traversal	Relationship, multi-hop and aggregate questions	High (extraction + graph DB)
RAG + evaluation	Metrics such as faithfulness and context precision	Proving quality to clients; monitoring	Medium

7. Q&A highlights

Question	Answer
Does Graph RAG still chunk and embed?	In the hybrid design, yes: documents are chunked and embedded into the vector index, **and** entities and relationships are extracted into the graph in parallel. Data is stored twice.
Does Neo4j store embeddings too?	It can: Neo4j has built-in vector indexes, so one database can hold both graph and vectors. The class design kept vectors in Azure AI Search.
How does the LLM query the graph?	It writes **Cypher** (a SQL-like graph query language), Neo4j runs it, and the results go back to the LLM to phrase the answer.
Is a Neo4j graph like LangGraph's nodes and edges?	The vocabulary is similar but the purpose differs. LangGraph models **agent workflow steps**; Neo4j stores **data** entities and relationships. LangGraph can't replace a graph database.
Alternatives to Neo4j?	Azure Cosmos DB for Apache Gremlin, Amazon Neptune, Memgraph, PostgreSQL + Apache AGE, or Microsoft's GraphRAG library (Parquet outputs + vector store).
Should I move to CRAG or Graph RAG instead of fine-tuning?	They solve different problems. Pick the RAG type from the questions; fine-tune only if the chosen RAG still underperforms.
Is CRAG the same as evaluation?	No. CRAG corrects at query time; evaluation measures quality.
Can Graph RAG still hallucinate?	Yes. Any LLM can. Ground the final prompt in graph results and evaluate it.
Can I put Graph RAG on my CV?	The lecturer encouraged it, once you've actually built and can explain it.
What's next?	Graph RAG re-run on a paid subscription, CRAG code, then MCP, A2A, tool and function calling, guardrails, then the capstone.

8. Expert corrections to the lecture

1. **Role-based access can't rely on prompt instructions alone.** Prompts can be bypassed by jailbreaks. Authenticate users (Entra ID) and enforce access in retrieval (e.g. Azure AI Search **security trimming** with group/role filters, row-level security in databases), so restricted data never reaches the LLM. Then add prompt guardrails on top.
2. **Cosmos DB is not for storing API keys.** Keys belong in **Key Vault**. Cosmos DB holds app data and chat history, and its Gremlin API can even serve as a graph database.
3. **Neo4j is a graph database,** not "the leader for relational databases" or an extension of SQL.
4. **"No chunking in Graph RAG" is too strong.** Microsoft GraphRAG chunks text, then runs LLM entity and relationship extraction per chunk, builds communities and writes summaries. Cypher is for querying the graph, not instead of extraction.
5. **Microsoft GraphRAG adds community summaries** (Leiden clustering) for **global** questions ("what are the main themes across all reviews?"), plus **local** search for entity-specific questions. That's a key interview point the lecture didn't mention.
6. **"Which hotels are near Creek Hotel?" isn't a multimodal-RAG question.** It needs location relationships (graph) or geo metadata filters.
7. **CRAG doesn't compute evaluation metrics or check guardrails internally.** Its evaluator grades retrieved documents (correct / incorrect / ambiguous) and triggers refinement or extra search. Context precision, recall and hallucination scores belong to evaluation frameworks.
8. **CRAG doesn't guarantee 99.9% accuracy.** It reduces errors from bad retrieval; regulated domains still need evaluation, guardrails and human review.
9. **Google Colab *can* connect to Azure, AWS and Neo4j Aura** over their public APIs (store keys in Colab Secrets). It only can't reach resources behind private networks. VS Code was a choice, not a requirement.
10. **Neo4j credentials:** the URI looks like `neo4j+s://<id>.databases.neo4j.io` and the username is `neo4j`. The instance ID isn't the username; the lecture's "username is nothing, it's called URI" was garbled.
11. **The 404 "Resource not found" error** usually means a wrong deployment name, endpoint path or API version, or a stale `.env`. It isn't caused by the code, and not necessarily by the free tier. Insufficient quota and deprecated models are separate errors.
12. **History:** Google launched in 1998; the June 2000 release announced a billion-URL index. PageRank is Google's algorithm, not something Yahoo and Netscape adopted in 2008. Knowledge graphs are old, but **GraphRAG** dates from Microsoft Research in 2024.

13. **Cypher** is Neo4j's declarative graph query language (openCypher); **GQL** became the ISO standard graph query language in 2024.
14. **Tuples:** immutability is a Python property. In the prompt, "tuple" just means a fixed (entity, relation, entity) format.

9. Interview quick revision

> **What is Graph RAG and when would you use it?** RAG whose retrieval traverses a knowledge graph of entities and relationships, built by LLM extraction from documents and stored in a graph DB like Neo4j. I use it when questions depend on relationships or multi-hop reasoning ("who is X's skip-level manager?", "which hotels did this reviewer rate?") or need global summaries across a corpus. Typically it's hybrid: vector search for passages plus Cypher queries for relationships, both fed to the LLM.

> **What is Corrective RAG?** RAG with a retrieval evaluator. After retrieving, it grades documents as correct, incorrect or ambiguous, refines the good ones, and re-queries or searches elsewhere for the bad ones, then generates. It reduces hallucinations from poor or stale retrieval and suits high-stakes domains, at the cost of extra LLM calls.

> **How would you choose between RAG types?** From the questions: direct facts → traditional; images and tables → multimodal; relationships → Graph RAG; high accuracy needs → CRAG; multi-tool, multi-step → agentic. Evaluate whichever you choose, and fine-tune only if it still falls short.

Practice

- [ ] Create a free Neo4j Aura instance and save the credentials file
- [ ] Run the Section 5 code with your own Foundry deployment; inspect the graph in Aura
- [ ] Write five relationship questions plain RAG fails on, and compare answers
- [ ] Sketch a CRAG flow in LangGraph: retrieve → grade → rewrite/search → generate